

Кафедра Информационные системы

К ЗАЩИТЕ ДОПУСТИТЬ

Зав. кафедрой

 /А. А. Романов /
подпись инициалы, фамилия

« » 20 г.

БАКАЛАВРСКАЯ РАБОТА

Вид ВКР (дипломный проект (работа) / бакалаврская работа / магистерская диссертация)

Тема Разработка подсистемы анализа кадровых данных для Ситуационного центра Губернатора Ульяновской области с использованием платформы Visiology

Обозначение ВКР БР-УЛГТУ-09.03.03-21/897-2025 Группа ИСЭбд-41
для технических направлений подготовки (специальностей)

Направление подготовки (специальность) 09.03.03 «Прикладная информатика»
код, наименование

Нормоконтроль _____ / Ю. В. Строева /
подпись, дата инициалы, фамилия

УЛЬЯНОВСК
2025 г.

	МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
	федеральное государственное бюджетное образовательное учреждение высшего образования
	«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет ИСТ

Кафедра Информационные системы

Направление подготовки (специальность) 09.03.03 «Прикладная информатика»

УТВЕРЖДАЮ

Зав. кафедрой

_____ / А. А. Романов /
подпись инициалы, фамилия

« _____ » _____ 2025 г.

ЗАДАНИЕ

на БАКАЛАВРСКУЮ РАБОТУ

указать вид ВКР (дипломный проект (работа) / бакалаврская работа / магистерская диссертация)

студенту Захарову Владиславу Витальевичу курса 4 группы ИСЭбд-41
фамилия, имя, отчество

Тема ВКР Разработка подсистемы анализа кадровых данных для Ситуационного центра Губернатора Ульяновской области с использованием платформы Visiology

утверждена приказом по УлГТУ от « 23 » _____ января _____ 2025 г. № 138

Срок сдачи обучающимся законченной ВКР _____

Исходные данные к ВКР Для Ситуационного центра Губернатора Ульяновской области необходимо разработать систему хранения и анализа кадровых данных на платформе Visiology

Содержание расчетно-пояснительной записки (перечень подлежащих разработке вопросов)

- 1) Анализ предметной области _____
- 2) Проектирование информационной системы _____
- 3) Реализация проекта _____
- 4) Тестирование проекта _____
- 5) Оценка эффективности системы _____

Перечень графического материала (с точным указанием обязательных чертежей)

1) Диаграмма «Как есть»

2) Диаграмма «Как должно быть»

3) Диаграмма прецедентов

4) ER-диаграмма

5) Диаграмма классов

6) Диаграмма компонентов

7) Диаграмма развертывания

8) Диаграмма взаимодействия

Календарный график работы над ВКР на весь период (с указанием сроков выполнения и содержания отдельных этапов)

№ этапа	Содержание этапа	Срок выполнения
1	Анализ требований	20.01.2025 - 21.02.2025
2	Проектирование архитектуры	22.02.2025 - 10.03.2025
3	Реализация системы	11.03.2025 - 20.03.2025
4	Тестирование	21.03.2025 - 31.03.2025
5	Защита и документирование	01.04.2025 - 06.04.2025

Дата выдачи задания «23» января 2025 г.

Руководитель _____ / В. В. Шеркунов /
должность, учёная степень, ученое звание подпись инициалы, фамилия

Задание принял к исполнению _____ / В. В. Захаров /
подпись обучающегося инициалы, фамилия

АННОТАЦИЯ

Аннотация

Тема Разработка подсистемы анализа кадровых данных для Ситуационного центра Губернатора Ульяновской области с использованием платформы Visiology.

Объем дипломной работы: 68 страниц (без приложения), в которых имеется 32 таблиц и 22 рисунка.

Цель дипломного проекта: путем разработки централизованного хранилища, автоматизировать процесс обработки информации, значительно сократить время на анализ и подготовку отчетности а.

Бакалаврская работа содержит в себе следующие разделы:

1. Анализ предметной области и формирование требований к разрабатываемой системе. В данном разделе формулируются цель и задачи проекта, описываются информационные процессы в Ситуационном центре Губернатора Ульяновской области, формулируются предложение по решению поставленной задачи, производится оформление диаграмм «Как есть» и «Как должно быть»;

2. Проектирование подсистемы анализа и подсистемы хранения кадровых данных для СЦ ГУО. В этом разделе содержится описание функциональной структуры приложения, разрабатываются диаграммы состояния и компонентов. Производится описание программного и технического обеспечения;

3. Реализация проекта. В данном разделе приводится контрольный пример и описание методик и протоколов тестирования проекта;

4. Оценка эффективности системы. Этот раздел содержит в себе анализ качества проектного решения и расчёт экономических результатов от внедрения.

СОДЕРЖАНИЕ

Аннотация	4
Введение.....	7
Раздел 1. Анализ предметной области и формирование требований к разрабатываемой системе.....	9
1.1. Описание предметной области.....	9
1.1.1. Описание объекта исследования, для которой разрабатывается система.	9
Экономическая характеристика:	9
1.1.2. Описание информационных/прикладных процессов исследования.....	10
1.2. Описание проблемы и анализ решений	11
1.2.1. Анализ проблемы существующих прикладных и информационных процессов.....	11
1.2.2. Состояние и стратегия развития схожих решений.....	11
1.2.3. Формирование предложений по разработке системы	12
1.3. Постановка задачи.....	16
1.3.1. Цели и задачи проекта	16
1.3.2. Требования к функциям системы.....	17
1.3.3. Спецификация и обоснование требований	18
1.4. Календарно-ресурсное планирование проекта, анализ бюджетных ограничений и рисков.....	19
Раздел 2. Проектирование подсистемы анализа и подсистемы хранения кадровых данных для СЦ ГУО	22
2.1. Диаграмма прецедентов системы	22
2.2. Инфологическая модель системы.....	26
2.3. Структурное описание проектируемой системы	29
2.3.1. Диаграмма классов.....	29
2.3.2. Диаграмма компонентов.....	34
2.3.3. Диаграмма развертывания.....	37
2.4. Описание взаимодействия прецедентов системы	39
Раздел 3. Реализация проекта.....	42
3.1. Контрольный пример работы с программой	42
3.1.1. Пример работы с порталом Visiology.	42

3.1.2. Пример работы с приложением обработки данных.	44
3.2. План тестирования	51
3.3. Протокол тестирования.....	52
3.3.1. Функциональное тестирование	52
3.3.2. Тестирование удобства пользования.....	55
3.3.3. Тестирование совместимости	57
3.3.4. Тестирование производительности	58
Раздел 4. Оценка эффективности системы	59
4.1. Расчет трудоемкости работы по разработке системы.....	59
4.2. Анализ затрат на ресурсное обеспечение	62
4.2.1. Расчет затрат на работу по разработке системы	62
4.2.2. Расчет стоимости эксплуатации оборудования.....	64
4.3. Анализ качественных и количественных факторов воздействия проекта на бизнес-архитектуру организации.....	65
4.3.1. Расчет экономических результатов от внедрения.....	65
4.3.2. Расчет сроков окупаемости проекта.....	65
Заключение	66
Список литературы	67
Приложение А	69

Введение

На данный момент кадровая аналитика в государственных структурах часто основывается на разрозненных данных, обработка которых требует значительных временных и трудовых затрат.

Ситуационный центр Губернатора Ульяновской области нуждается в инструменте, который позволит оперативно анализировать кадровые данные.

Разработка системы анализа кадровых данных позволит автоматизировать процесс обработки информации и значительно сократить время на анализ и подготовку отчетности [11].

Таким образом, создание подсистемы анализа кадровых данных является актуальной задачей, способствующей цифровизации кадровых процессов и повышению эффективности работы Ситуационного центра Губернатора Ульяновской области.

Объектом данного исследования являются процессы визуализации кадровых данных.

Предмет исследования - автоматизация процесса обработки кадровой информации.

Целью данной работы является разработка централизованного хранилища, которое позволит автоматизировать процесс обработки информации, сократив количество ошибок при работе с данными и значительно сократить время на анализ и подготовку отчетности.

Для достижения данной цели необходимо решить несколько задач:

1. Разработать структуру базы данных
2. Разработать макет пользовательского интерфейса для подсистемы хранения данных и согласовать его с заказчиком
3. Разработать подсистему хранения кадровых данных на базе Python
4. Создать макет подсистемы анализа и согласовать его с заказчиком
5. Разработать подсистему анализа кадровых данных на базе Dashboard Designer от Visiology

6. Внедрить проект

Методы, технологии, инструментарий проведения работы.

- Анализ требований
- Анализ схожих решений
- Проектирование системы
- Разработка системы
- Тестирование программного продукта
- Анализ качества проектного решения

Результатом выполнения выпускной квалификационной работы является разработанная централизованного хранилища данных и подсистемы визуализации данных.

Раздел 1. Анализ предметной области и формирование требований к разрабатываемой системе

1.1. Описание предметной области

1.1.1. Описание объекта исследования, для которой разрабатывается система.

Кадровый отдел администрации Губернатора Ульяновской области, а именно его статистические данные, в данном проекте являются объектом исследования. Данные охватывают различные аспекты кадрового состава администрации (гендерный, возрастной составы, наставничество, образование служащих и др.).

В качестве объекта автоматизации рассматривается процесс ежегодной отчетности отдела кадров, осуществляемая на плановых совещаниях.

Экономическая характеристика:

1. Правильная и понятная визуализация информации позволит вышестоящим руководителям проще принимать нужные управленческие решения.
2. Значительное сокращение времени на подготовку отчетности.
3. Регион активно внедряет цифровые технологии в рамках нацпроекта «Цифровая экономика»;
4. Существует потребность в повышении эффективности управления за счёт инструментов Business Intelligence (BI), таких как платформа Visiology.

1.1.2. Описание информационных/прикладных процессов исследования.

Анализ текущих прикладных и информационных процессов:

В текущем состоянии процессы сбора, анализа данных и отчетности реализуются в ручном режиме, посредством агрегации данных в Excel таблицах и отчетности при помощи презентаций PowerPoint. Текущий процесс описывается в таблице 1 и рисунке 1.

Таблица 1 - Роли в системе

Роль	Основные задачи
Сотрудники отдела кадров	Сбор необходимых данных
Сотрудники ситуационного центра	Подготовка и загрузка данных в систему

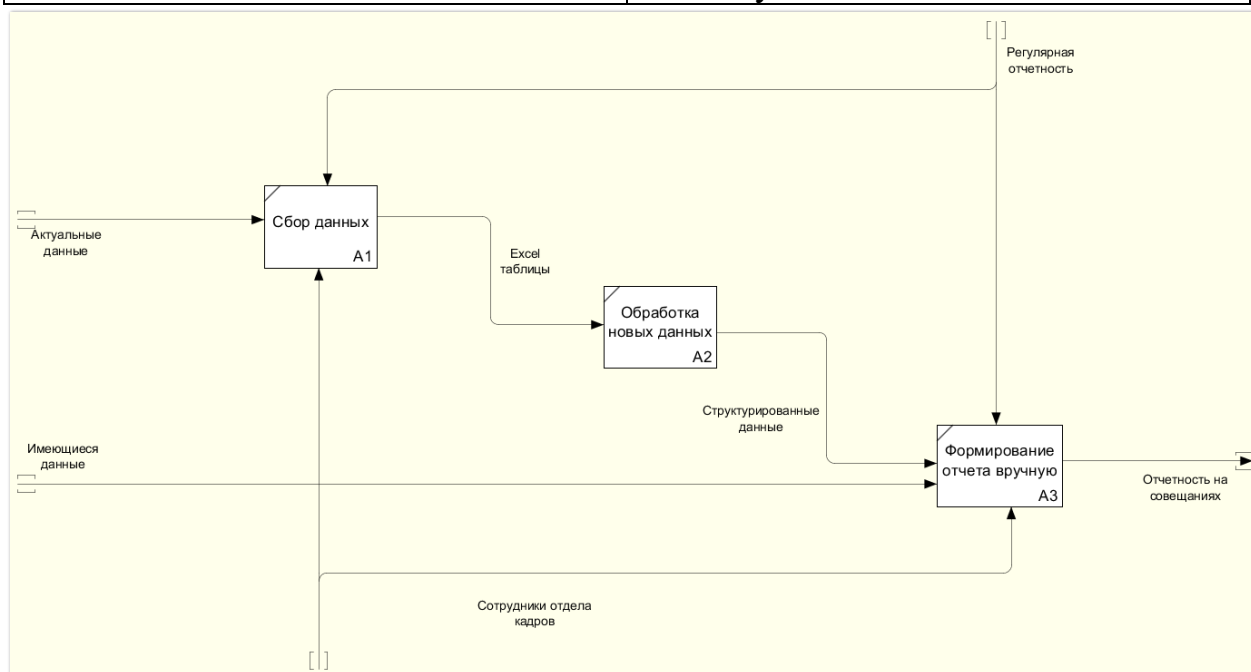


Рисунок 1 - Модель IDEF0 «Как есть»

1.2. Описание проблемы и анализ решений

1.2.1. Анализ проблемы существующих прикладных и информационных процессов

На текущий момент информационные процессы в кадровом отделе администрации ГУО характеризуются следующими проблемами [15]:

1. Данные хранятся в табличной форме (Excel), без централизованного доступа.
2. Ручной сбор и обработка данных, при котором есть высокая вероятность ошибок. Высокая трудоёмкость, отсутствие единой формы подачи информации также мешают.
3. Отсутствие инструментов динамической аналитики, из-за чего невозможность оперативно анализировать данные в реальном времени.
4. Ограниченные возможности визуализации

1.2.2. Состояние и стратегия развития схожих решений

На федеральном уровне и в других регионах реализуются аналогичные цифровые решения, ориентированные на анализ больших массивов данных. Наиболее заметные инициативы:

- Цифровая платформа "Цифровое сельское хозяйство" Минсельхоза РФ — федеральный уровень мониторинга и управления.
- BI-решения в других регионах: например, в Белгородской, Краснодарской и Воронежской областях используются специализированные панели мониторинга, построенные на таких платформах, как Power BI, Qlik и Visiology.

Стратегия развития похожих решений строится на:

- Централизации хранения данных;
- Интерактивном доступе к аналитике;
- Упрощении пользовательского интерфейса.

1.2.3. Формирование предложений по разработке системы

На основе анализа существующих решений и выявленных проблем предлагается разработать подсистему анализа кадровых данных на базе платформы Visiology и подсистему хранения данных.

Ключевые предложения:

Выбор платформы Visiology обусловлен следующими факторами:

- Используется у заказчика в данный момент
- Поддержка гибкой визуализации (дашборды, графики, карты, KPI);
- Возможность интеграции с разными источниками данных (Excel, SQL, API);
- Соответствие требованиям по защите информации (возможность развёртывания в закрытом контуре);
- Поддержка русскоязычного интерфейса и соответствие нормативам РФ.

Учет успешных ИТ-проектов:

- Внедрение интерактивных панелей мониторинга (с функциями Drill Down и подобными) с фильтрами по субъектам федерации и возможностью выгрузить выбранные данные;
- Применение аналитики в реальном времени

Анализ рынка программного обеспечения:

Среди BI-платформ присутствуют: Microsoft Power BI, Yandex DataLens, Tableau, Visiology.

Visiology выигрывает за счет локализации, поддержки российских стандартов, возможности работы в защищённых средах и конкурентной стоимости. Сравнение аналогов представлены в таблице 2.

Таблица 2 - Сравнение аналогов разрабатываемой системы

	Доступность	Продвинутая визуализация	Сложность освоения	Зависимость от собственной экосистемы
Tableau	Нет	Да	Сложно	Нет
Microsoft Power BI	Нет	Да	Просто	Да
Yandex DataLens	Да	Нет	Просто	Да
Visiology	Да	Да	Средне	Нет

1) Tableau

Преимущества:

- Продвинутая визуализация – позволяет создавать сложные, интерактивные и наглядные дашборды.
- Гибкая работа с данными – поддержка множества источников данных, возможность подключения к большим базам и обработка в реальном времени.
- Высокая производительность – быстро обрабатывает большие объемы данных.
- Интуитивный интерфейс – удобный drag-and-drop функционал для построения графиков и отчетов.
- Широкий функционал аналитики – мощные инструменты для продвинутого анализа данных, включая прогнозирование и кластеризацию.

Недостатки:

- Сложность освоения – требует времени для обучения, особенно при работе со сложными моделями данных.
- Ограниченная интеграция с Microsoft-продуктами – хуже взаимодействует с Excel, SharePoint и Power BI-сервисами.
- Требовательность к ресурсам – на слабых ПК и при обработке больших данных возможны задержки.
- Проблемы с доступностью – не распространяется на территории РФ

2) Microsoft Power BI

Преимущества:

- Доступность и цена – есть бесплатная версия, а платные лицензии стоят дешевле, чем у конкурентов.
- Интеграция с Microsoft – легко подключается к Excel, Azure, SQL Server и другим продуктам Microsoft.
- Простота освоения – интуитивно понятный интерфейс, особенно для пользователей Excel.
- Мощные возможности работы с облаком – поддержка Microsoft Azure, автоматическое обновление данных.
- Гибкие варианты развертывания – работает как в облаке, так и на локальных серверах.

Недостатки:

- Ограничения в обработке больших данных – при работе с объемными наборами данных возможны задержки.
- Зависимость от экосистемы Microsoft – интеграция с другими сервисами (кроме Microsoft) может быть сложной.
- Проблемы с доступностью – не распространяется на территории РФ

Yandex DataLens

Преимущества:

- Бесплатный базовый тариф – позволяет использовать DataLens без дополнительных затрат (с ограничениями по количеству запросов).
- Интеграция с экосистемой Яндекса – удобная работа с Yandex Cloud, Yandex Data Streams, Yandex ClickHouse и другими сервисами.
- Облачное решение – не требует установки на локальные серверы, доступен через браузер.
- Поддержка различных источников данных – SQL-базы (PostgreSQL, ClickHouse, MySQL), Google Sheets, 1С и другие.
- Интуитивно понятный интерфейс – простая настройка дашбордов, фильтров и визуализаций.
- Возможность работы в закрытом контуре – поддержка развёртывания в частных облаках для бизнеса и госсектора.

Недостатки:

- Ограниченные возможности кастомизации – мало инструментов для сложных визуализаций.
- Зависимость от облачной инфраструктуры Яндекса – менее удобна при использовании других облачных платформ.
- Ограниченный офлайн-доступ – так как это облачный сервис, без интернета работать с дашбордами невозможно.

Visiology

Преимущества:

- Наличие – уже имеется у заказчика.
- Полностью российская разработка – подходит для импортозамещения и соответствует требованиям по безопасности данных.
- Гибкая визуализация – мощный инструмент для построения дашбордов, отчётов и аналитических панелей.
- Интеграция с различными источниками данных – поддержка SQL-баз, 1С, Excel, API и других систем.

- Высокая производительность – оптимизирован для работы с большими массивами данных.
- Возможность работы в закрытых контурах – важно для госсектора и корпоративных пользователей.
- Локальная и облачная версия – возможность развернуть как в частном облаке, так и на собственных серверах.

Недостатки:

- Высокий порог вхождения – требует времени на освоение, особенно для пользователей без опыта работы с BI-системами.
- Зависимость от российского рынка – меньшая известность и комьюнити по сравнению с Tableau и Power BI, что может ограничивать доступ к ресурсам и обучающим материалам.
- Сложность развертывания – для корпоративных внедрений может требовать сложной настройки и технической поддержки.

Выбор технологии проектирования и разработки:

- Язык описания бизнес-процессов: BPMN / IDEF0;
- Архитектурный подход: модульно-компонентный;
- Технологии хранения данных: PostgreSQL, ViQube;
- Интерфейс взаимодействия: веб-приложение подсистемы аналитики данных, встроенное в корпоративный портал или портал органов власти; desktop-приложение подсистемы хранения данных, устанавливаемое на ПК сотрудников.

1.3. Постановка задачи

1.3.1. Цели и задачи проекта

Целью проекта является разработка централизованного хранилища, которое позволит автоматизировать процесс обработки информации, сократить количество ошибок при работе с данными и значительно сократить время на анализ и подготовку отчетности.

Задачи проекта:

1. Разработать структуру базы данных
2. Создать и заполнить тестовыми данными базу данных
3. Разработать макет пользовательского интерфейса для подсистемы хранения данных и согласовать его с заказчиком
4. Разработать подсистему хранения кадровых данных на базе Python
5. Создать макет подсистемы анализа для согласования его с заказчиком
6. Разработать подсистему анализа кадровых данных на базе Dashboard Designer от Visiology
7. Внедрить проект (получить акт о внедрении программных средств у заказчика)

Окружение модуля:

Вход: внешние базы данных (отчетность сельхозпредприятий, метеослужбы, Росстат и др.), внутренние ведомственные таблицы;

Выход: визуальные отчеты, графики, карты, аналитические панели;

Интеграция: возможна с другими ведомственными ИС (например, 1С, внутренние базы данных органов власти).

1.3.2. Требования к функциям системы

Процесс работы после внедрения системы представлен на рисунке 2.

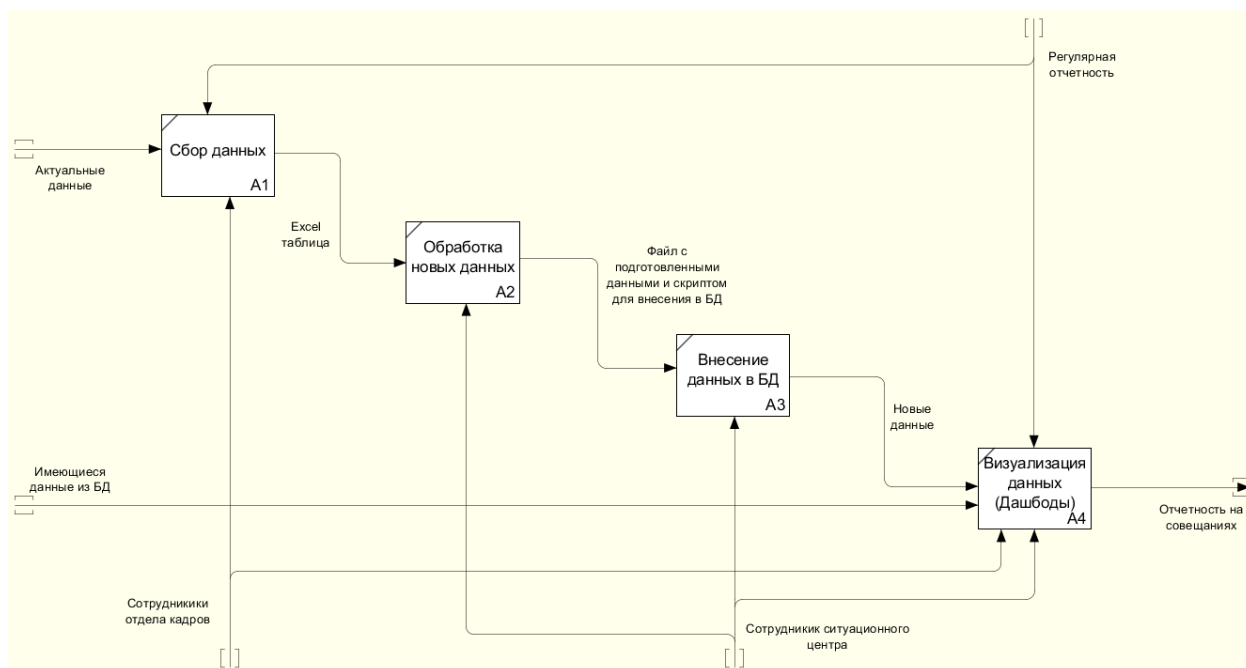


Рисунок 2 - Модель IDEF0 «Как должно быть»

Сводная таблица функциональных требований

Функциональные требования системы представлены в таблице 3.

Таблица 3 - Функциональные требования

№	Функция	Вход	Выход	Участники
1	Сбор данных	Актуальные данные	Excel таблица	Сотрудники отдела кадров
2	Обработка новых данных	Excel таблица	Файл с готовыми данными	Система, администратор БД
3	Внесение данных в БД	Файл с готовыми данными	Новые данные	Сотрудники СЦ
4	Визуализация данных (Дашборды)	Новые данные, имеющиеся в БД данные	Отчетность на совещаниях	Сотрудники СЦ, Сотрудники отдела кадров

1.3.3. Спецификация и обоснование требований

Требования к программно-технической среде. Сводная таблица:

Требования к программно-технической среде представлены в таблице 4.

Таблица 4 - Требования к программно-технической среде

Компонент	Характеристика/Обоснование
Операционная система	Windows 10 или Astra Linux – соответствие стандартам безопасности и стабильности

BI-Платформа	Visiology Platform (ViQube, SmartForms, Dashboards) – поддержка визуализации, фильтрации, построения дашбордов
СУБД	PostgreSQL / MS SQL Server – масштабируемость, высокая производительность при работе с большими объемами данных
ETL-инструменты	Встроенные в платформу Visiology или внешние (например, Python-скрипты)
Архитектура	Клиент-сервер, с возможностью размещения в изолированной сетевой инфраструктуре учреждения
Аппаратное обеспечение	Сервер с минимум 16 ГБ ОЗУ, CPU от 8 ядер, SSD-накопители от 500 ГБ

Пользовательские требования:

Пользовательские требования к системе представлены в таблице 5.

Таблица 5 - Пользовательские требования

Категория	Требование	Обоснование
Быстродействие	Время загрузки аналитической панели – не более 5 секунд	Комфорт пользователя при работе с визуализацией
Информационная безопасность	Авторизация по ролям	Обработка государственной отчетности
Масштабируемость	Возможность добавления новых источников, показателей, отчётных форм	Развитие функционала по мере появления новых требований
Интеграция	Возможность обмена данными с внешними системами через API и таблицы	Синхронизация с другими ИС ведомства

1.4. Календарно-ресурсное планирование проекта, анализ бюджетных ограничений и рисков

Модель жизненного цикла разработки информационной системы

В рамках реализации проекта используется каскадная (Waterfall) модель жизненного цикла разработки. Этот подход обоснован структурированным характером задачи и наличием чётких требований [4].

Этапы жизненного цикла:

1. Анализ и сбор требований
2. Проектирование архитектуры модуля и структуры БД
3. Разработка и тестирование подсистемы анализа данных
4. Разработка и тестирование подсистемы хранения данных
5. Тестирование и валидация
6. Ввод в эксплуатацию
7. Обучение пользователей и сопровождение

Анализ ограничений и рисков

Бюджетные ограничения:

- Использование только сертифицированного ПО (Visiology, Astra Linux) может ограничивать выбор технологий.
- Требуется соблюдение ФЗ о госзакупках при привлечении сторонних разработчиков.
- Потенциальная необходимость дополнительного серверного оборудования.

Риски проекта и пути их минимизации:

Возможные риски проекта представлены в таблице 6.

Таблица 6 - Риски проекта

Риск	Вероятность	Влияние	Меры реагирования
Отсутствие доступа к актуальным данным	Низкая	Высокое	Согласование с владельцами ИС на раннем этапе
Низкая ИТ-компетенция пользователей	Высокая	Среднее	Проведение обучающих сессий, разработка инструкций

Календарный план:

Календарный план [17] работы представлен в таблице 7.

Таблица 7 - Календарный план

№	Содержание этапа	Срок выполнения
1	Анализ требований	20.01.2025 - 21.02.2025
2	Проектирование архитектуры	22.02.2025 - 10.03.2025
3	Реализация системы	11.03.2025 - 20.03.2025

4	Тестирование	21.03.2025 - 31.03.2025
5	Защита и документирование	01.04.2025 - 06.04.2025

На основании данных исследований, можно сделать вывод, что разрабатываемый продукт уникален, актуален, а главное необходим.

Раздел 2. Проектирование подсистемы анализа и подсистемы хранения кадровых данных для СЦ ГУО

2.1. Диаграмма прецедентов системы

Диаграмма прецедентов (Use-Case Diagram) — диаграмма, описывающая цели пользователей, взаимодействие между пользователями и системой и требуемое поведение системы для удовлетворения этих целей [7].

Модель вариантов использования состоит из ряда модельных элементов (или графических примитивов). К наиболее важным из них относятся варианты использования (use cases), действующие лица или акторы (actors) и отношения (связи) между ними (relationships) [12].

В ходе анализа предметной области были определены акторы системы и перечень вариантов использования.

Описание диаграммы прецедентов представлены в таблицах 8 и 9.

Таблица 8 - Список акторов системы

Идентификатор	Действующее лицо
АСТ-01	Оператор данных
АСТ-02	Аналитик

Таблица 9 - Перечень вариантов использования

Основной актор	Код	Наименование	Формулировка
АСТ-1	UC-1	Редактирование параметров форматирования	Этот вариант использования позволяет актору редактировать параметры форматирования файлов (добавлять/удалять листы, просматривать параметры, настройка списка игнорирования)

Основной актер	Код	Наименование	Формулировка
АСТ-1	UC-1.1	Добавить настройки листа	При нажатии на соответствующую кнопку открывается диалоговое окно для создания настройки листа.
АСТ-1	UC-1.2	Удалить настройки листа	При нажатии на соответствующую кнопку открывается диалоговое окно для удаления настройки листа.
АСТ-1	UC-1.3	Просмотр настроек	При нажатии на соответствующую кнопку открывается диалоговое окно, в котором показываются все настройки.
АСТ-1	UC-2	Форматирование исходных данных	Этот вариант использования позволяет актору обработать выбранный файл, и подготовить его к загрузке в БД
АСТ-1	UC-2.1	Настройка игнорирования листов	При нажатии кнопки «Игнорирование листов» на главной форме, открывается диалоговое окно, в котором можно выбрать листы, которые будут игнорироваться при форматировании.
АСТ-1	UC-2.2	Указать год файла	Перед началом форматирования нужно выбрать год файла, который будет указываться результате.
АСТ-1	UC-2.3	Указать вид файла	Перед началом форматирования нужно указать тип файла. Это влияет на способ форматирования.

Основной актер	Код	Наименование	Формулировка
АСТ-1	UC-3	Загрузка данных в БД	Этот вариант использования позволяет актору отправить обработанные данные в БД
АСТ-1	UC-4	Получить файл настроек	Этот вариант использования позволяет актору экспортировать файл настроек
АСТ-2	UC-5	Просмотр дашбордов	Этот вариант использования позволяет актору просматривать и анализировать данные
АСТ-2	UC-5.1	Выгрузка данных с дашборда	При нажатии на кнопку «Выгрузить» на дашборде, произойдет выгрузка данных выбранного виджета в формате Excel файла.
АСТ-2	UC-5.2	Выбор фильтров	На каждом листе дашборда есть возможность выбрать фильтры, которые будут влиять на показываемые данные
АСТ-2	UC-5.3	Загрузка данных из ViQube	Происходит автоматически при открытии дашборда и переходе между листами.

Описанная диаграмма прецедентов представлена на рисунке 3.

Диаграмма прецедентов

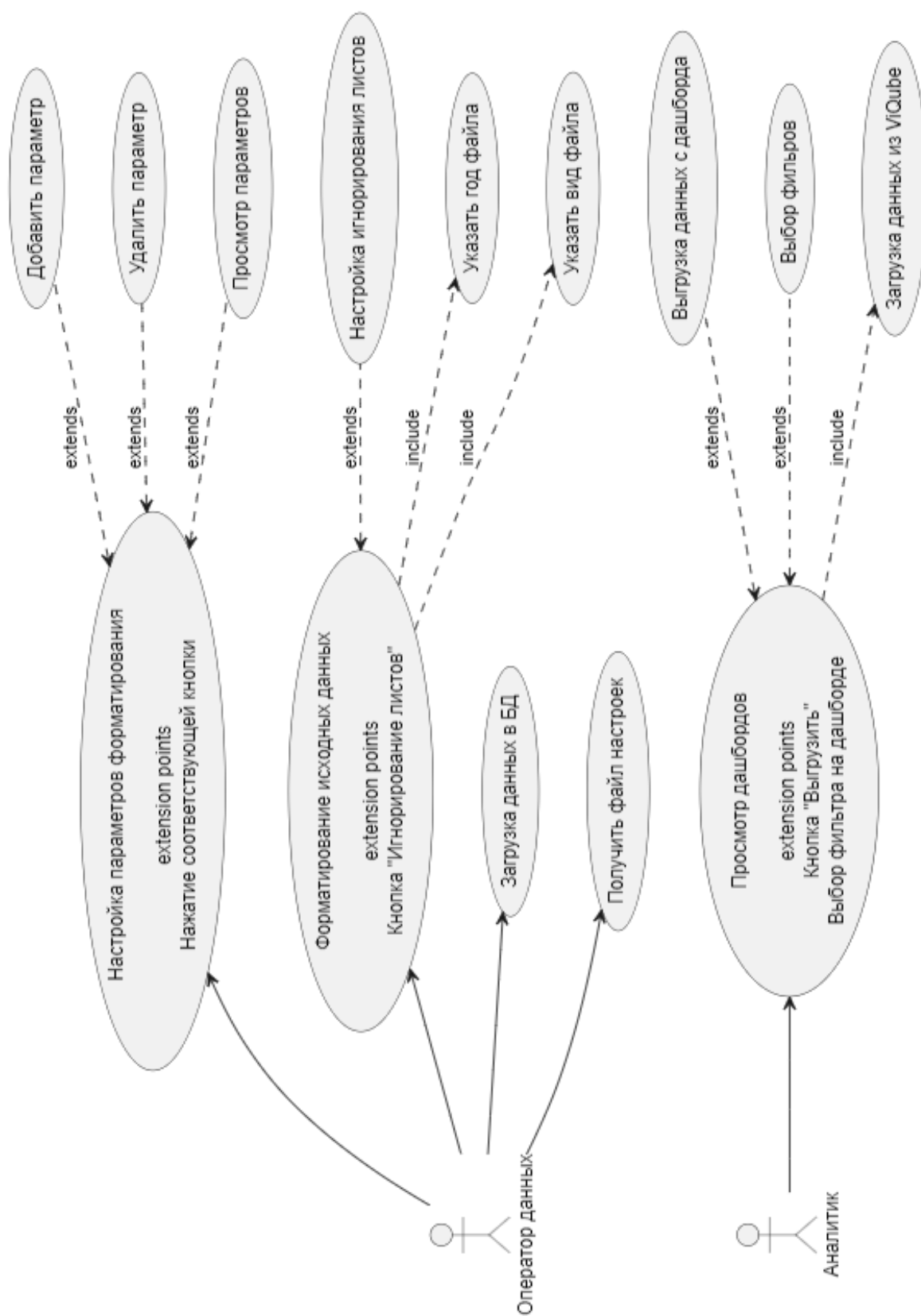


Рисунок 3 - Диаграмма прецедентов

2.2. Инфологическая модель системы

Для построения инфологической модели системы, была построена ER-диаграмма

ER-диаграмма или диаграмма сущность-связь (Entity-Relationship Diagram) — это инструмент моделирования данных, используемый в базах данных и системах управления информацией. Она позволяет визуализировать структуру базы данных, отображая сущности, их атрибуты и связи между ними [1]. ER-диаграммы применяются для проектирования реляционных баз данных и помогают разработчикам лучше понимать взаимосвязь различных элементов данных.

Описание ER-диаграммы представлено в таблицах 10, 11, 12 и 13.

Таблица 10 - SubFed (Субъекты федерации)

Поле	Тип данных	Описание
id	integer(10)	Первичный ключ
name	varchar(255)	Название субъекта федерации, Unique

Таблица 11 - Sheet (Лист исходного файла Excel)

Поле	Тип данных	Описание
id	integer(10)	Первичный ключ
name	varchar(255)	Название листа, Unique

Таблица 12 - Category (Категория или другой параметр, уровень мультииндекса)

Поле	Тип данных	Описание
id	integer(10)	Первичный ключ
text	TEXT - (тип данных в PostgreSQL для хранения строк произвольной длины)	Текст уровня мультииндекс

Таблица 13 - Kadr_data. Основная таблица, в которой хранятся все обработанные данные.

Поле	Тип данных	Описание
id	integer(10)	Первичный ключ
year	smallint(5)	Год
id_SubFed	integer(10)	Внешний ключ таблицы SubFed
id_Sheet	integer(10)	Внешний ключ таблицы Sheet
id_category_1	integer(10)	Внешний ключ таблицы Category
Id_category_2	integer(10)	Внешний ключ таблицы Category, Nullable
id_category_3	integer(10)	Внешний ключ таблицы Category, Nullable
value	integer(10)	Значение

Описанная ER-диаграмма представлена на рисунке 4.

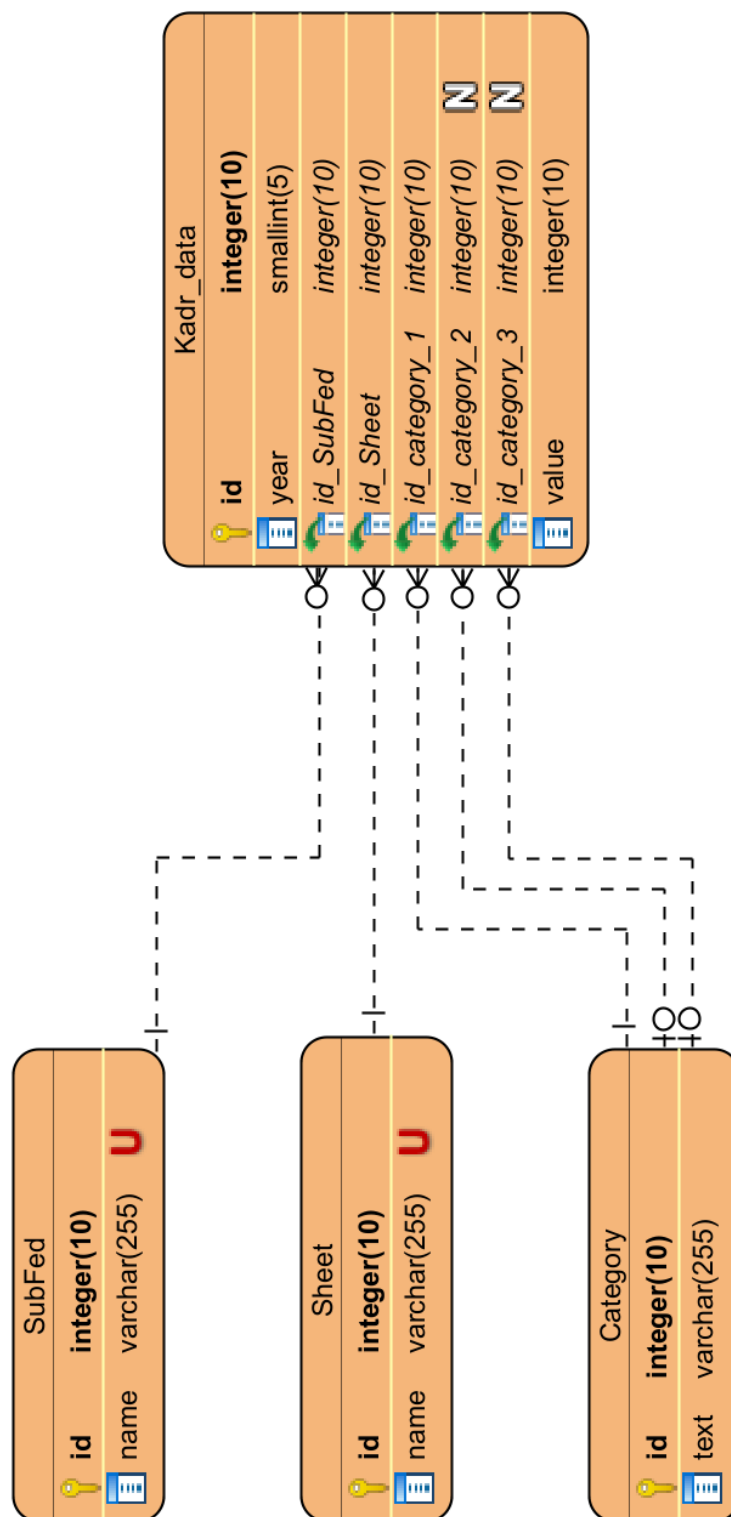


Рисунок 4 - ER-диаграмма

2.3. Структурное описание проектируемой системы

2.3.1. Диаграмма классов

Диаграмма классов (Class Diagram) представляет собой визуальное описание типов объектов в системе и статических связей между ними [7]. Она включает в себя характеристики классов, их методы и ограничения, действующие на взаимоотношения между объектами. Эта диаграмма разрабатывается для определения сущности предметной области и представить их в форме классов с соответствующими атрибутами и операциями, определить взаимосвязи между сущностями предметной области и представить их в форме типовых отношений между классами [5].

Описание классов:

Класс main

Функция get_mode() -> BaseFormatter

Возвращает объект класса BaseFormatter или его наследник в зависимости от выбранного на форме варианта

Процедура start_format_async()

Асинхронно запускает процесс обработки файла. Результатом работы является набор csv файлов с данными

Процедура update_status_label(text, btn_disabled)

Обновляет строку статуса на форме. Получает текст нового статуса (text), и новое состояние кнопки форматирования (btn_disabled).

Процедура open_file()

Вызывает диалоговое окно для выбора файла в системе

Процедура select_result_path()

Вызывает диалоговое окно для выбора пути сохранения результата обработки

Процедура open_window_settings_ignore()

Вызывает диалоговое окно для настройки списка игнорирования

Процедура save_ignore_list()

Формирует и сохраняет список игнорирования. Список игнорирования содержит в себе названия листов, которые нужно проигнорировать в процессе форматирования.

Процедура open_window_settings()

Открывает окно редактирования настроек

Процедура import_settings()

Вызывает диалоговое окно для выбора файла настроек и импортирует его в приложение

Процедура export_settings()

Вызывает диалоговое окно для выбора пути сохранения и экспортирует файл настроек в формате json

Функция send_to_DB()

Отправка последних обработанных данных в БД

Класс SettingsWindow

settingsManager: Settings

Параметр хранящий в себе функции работы с настройками

Процедура btn_add()

Вызывает диалоговое окно создания настройки листа

Процедура btn_delete()

Вызывает диалоговое окно удаления настройки листа

Процедура btn_show()

Вызывает диалоговое со всеми имеющимися настройками

Процедура new_sheet_save()

Создает и сохраняет новую настройку

Процедура delete_selected_sheet()

Удаляет выбранную настройку

Базовый класс BaseFormater

Процедура save_to_csv(dataframe: DataFrame, file_path: str, separator: str)

-> None

Сохраняет полученные dataframe в csv с разделителем separator файл по пути file_path

Функция data_frame_from_excel(path: str, settings: Setting) -> DataFrame

Получает файл по пути path и обрабатывает его по настройкам settings, возвращает DataFrame

Процедура start_format(sheet_settings: [str, Setting], file_path: str, file_year: int, save_path: str, ignore_sheet: set, excel_to_csv=None) -> None

Начинает процесс форматирования. Получает на вход sheet_settings – массив настроек, file_path – путь исходного файла, file_year – год файла, save_path – путь для сохранения результата, ignore_sheet – список листов, которые нужно игнорировать, excel_to_csv - ссылка на функцию для форматирования листа

Класс UlskFormatter

Функция excel_to_csv(path: str, year: int, settings: Setting) -> DataFrame

Получает на вход path – путь к файлу, year – год файла, settings – настройка форматирования листа. Обрабатывает файл и возвращает DataFrame

Класс FullFormatter

Функция excel_to_csv(path: str, year: int, settings: Setting) -> DataFrame

Получает на вход path – путь к файлу, year – год файла, settings – настройка форматирования листа. Обрабатывает файл и возвращает DataFrame

Класс SettingsViewModel

Процедура addSheet(string)

Создает и сохраняет настройку листа

Процедура removeSheet(string)

Удаляет настройку листа

Процедура ignoreSheet(string)

Сохраняет лист игнорирования настроек

Функция `getSettings()` -> str

Возвращает все настройки в виде строки str

Класс `Setting`

Класс модель для хранения и передачи настроек в приложении

sheet: str – название листа

iloc_rows: list[int] – срез по строкам

iloc_columns: list[int] – срез по столбцам

drop_column: list[int] – список столбцов, которые нужно пропустить

m_id_lists: list[list[str]] – уровни мультииндекса

m_id_names: list[str] – название уровней мультииндекса

csv_path: str – название конечного csv файла

Класс `BDConnection`

Процедура `connect()`

Устанавливает соединение с БД

Процедура `executeQuery(query)`

Осуществляет запрос в БД

Описанная диаграмма классов представлена на рисунке 5.

Диаграмма классов

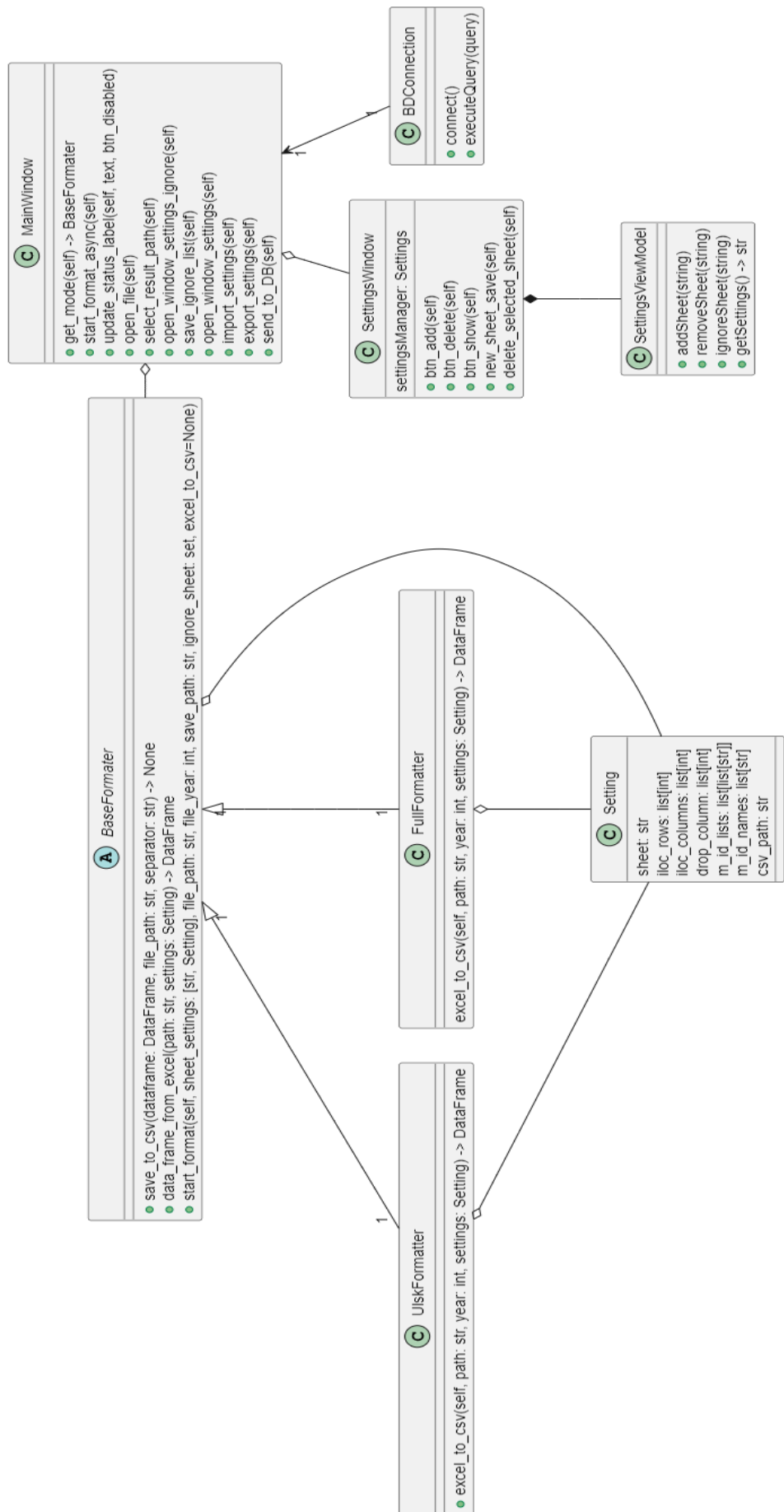


Рисунок 5 - Диаграмма классов

2.3.2. Диаграмма компонентов

Диаграмма компонентов отражает особенности физической структуры системы и помогает определить её архитектуру, устанавливая зависимости между программными компонентами [7]. Она используется для визуального представления общей организации исходного кода системы, отражая общие зависимости между компонентами, рассматривая последние в качестве классификаторов [20]. Спецификация компонентов в модели программной системы позволяет их повторно использовать в различных проектах.

Диаграмма компонентов состоит из:

Пакет «Приложение обработки данных», который состоит из 3 компонентов:

- **pandas** - библиотека для Python [16], которая упрощает работу с табличными данными: позволяет загружать, обрабатывать, анализировать и сохранять данные из различных источников (например, CSV или Excel) с помощью удобных структур — DataFrame и Series.
- **PySide6 GUI** – графический пользовательский интерфейс разработанный на PySide6 - официальной библиотеке Python для создания графических интерфейсов (GUI) с использованием фреймворка Qt 6 [3]. Она позволяет разрабатывать кроссплатформенные desktop-приложения с окнами, кнопками, таблицами, выпадающими списками и другими элементами интерфейса.
- **psycopg2** - библиотека для Python [16], которая позволяет подключаться к базе данных PostgreSQL, выполнять SQL-запросы и обрабатывать результаты.

Пакет «Сервер БД», который состоит из 2 компонентов:

- **PostgreSQL** - объектно-реляционная система управления базами данных (СУБД) с открытым исходным кодом [2]. Она поддерживает стандарт SQL, расширения, транзакции, работу с большими объёмами данных и сложные запросы, а также даёт возможность хранить и обрабатывать как

структурированные, так и частично структурированные данные (например, JSON).

- **ViQube** - высокопроизводительная in-memory база данных с поддержкой многомерной OLAP-модели, разработанная компанией Visiology. Она является ключевым компонентом аналитической платформы Visiology и предназначена для быстрой обработки больших объёмов данных, обеспечивая мгновенный доступ к аналитической информации.

Пакет «Портал Visiology», который состоит из 1 компонента:

- **Веб – приложение (Дашборд)** – этот компонент получает данные из ViQube через ViQubeAPI [8].

Описанная диаграмма компонентов представлена на рисунке 6.

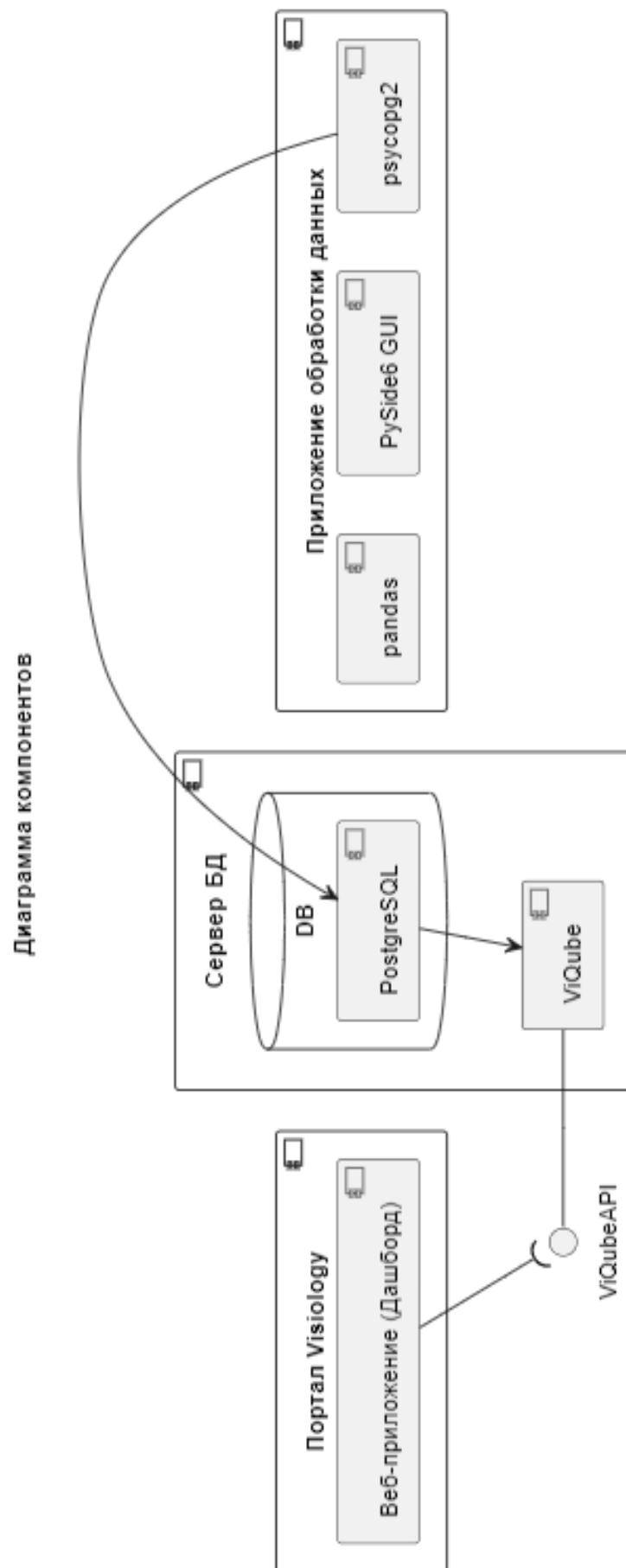


Рисунок 6 - Диаграмма компонентов

2.3.3. Диаграмма развертывания

Для графического представления узлов и взаимосвязей между ними предназначена диаграмма развертывания (Diagram Deployment) [7]. Диаграммы развертывания отображают физическую структуру системы, демонстрируя, на каком оборудовании размещаются и запускаются различные компоненты программного обеспечения. Она показывает, какие компоненты разработанного и используемого программного обеспечения на каких устройствах и в каких средах исполнения должны быть размещены [10].

В данном случае система для работы системы требуется персональный компьютер сотрудника (оператора данных), на котором будет установлено приложение обработки данных, и сервер, на котором будут расположены базы данных PostgreSQL и ViQube. Также на компьютере сотрудника должен быть установлен браузер для доступа к portalу Visiology.

Приложение обработки данных на ПК сотрудника и база данных PostgreSQL на сервере, связаны через библиотеку psycopg2.

Portal Visiology в браузере на ПК сотрудника и ViQube на сервере связаны через ViQubeAPI.

Описанная диаграмма развертывания представлена на рисунке 7.

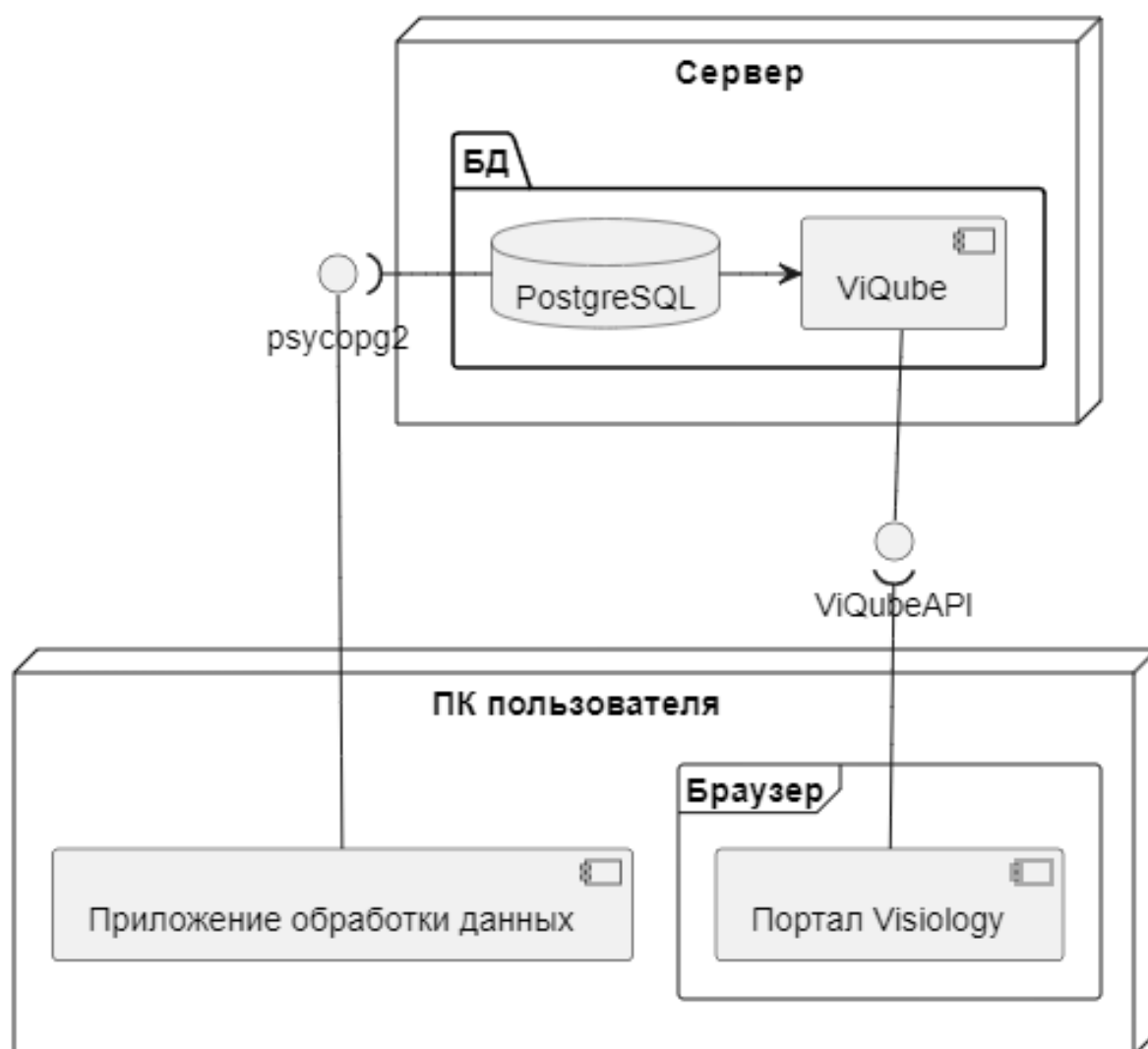


Рисунок 7 - Диаграмма развертывания

2.4. Описание взаимодействия прецедентов системы

Для описания взаимодействия прецедентов системы была построена диаграмма последовательности.

Диаграмма последовательности (Sequence Diagram) отображает взаимодействие между группами объектов в различных сценариях поведения [7]. На таких диаграммах присутствует временная шкала, которая условно направлена сверху вниз, отражая порядок обмена сообщениями во времени [9].

Диаграмма последовательности для сценариев работы с приложением обработки данных включает в себя последовательности для сценариев:

- Обработка исходных данных
- Отправка данных в БД
- Импортирование настроек
- Экспортирование настроек
- Редактирование настроек форматирования
- Просмотр настроек
- Добавление настройки листа
- Удаление настройки листа

Описанная Диаграмма последовательности представлена на рисунках 8 и 9.

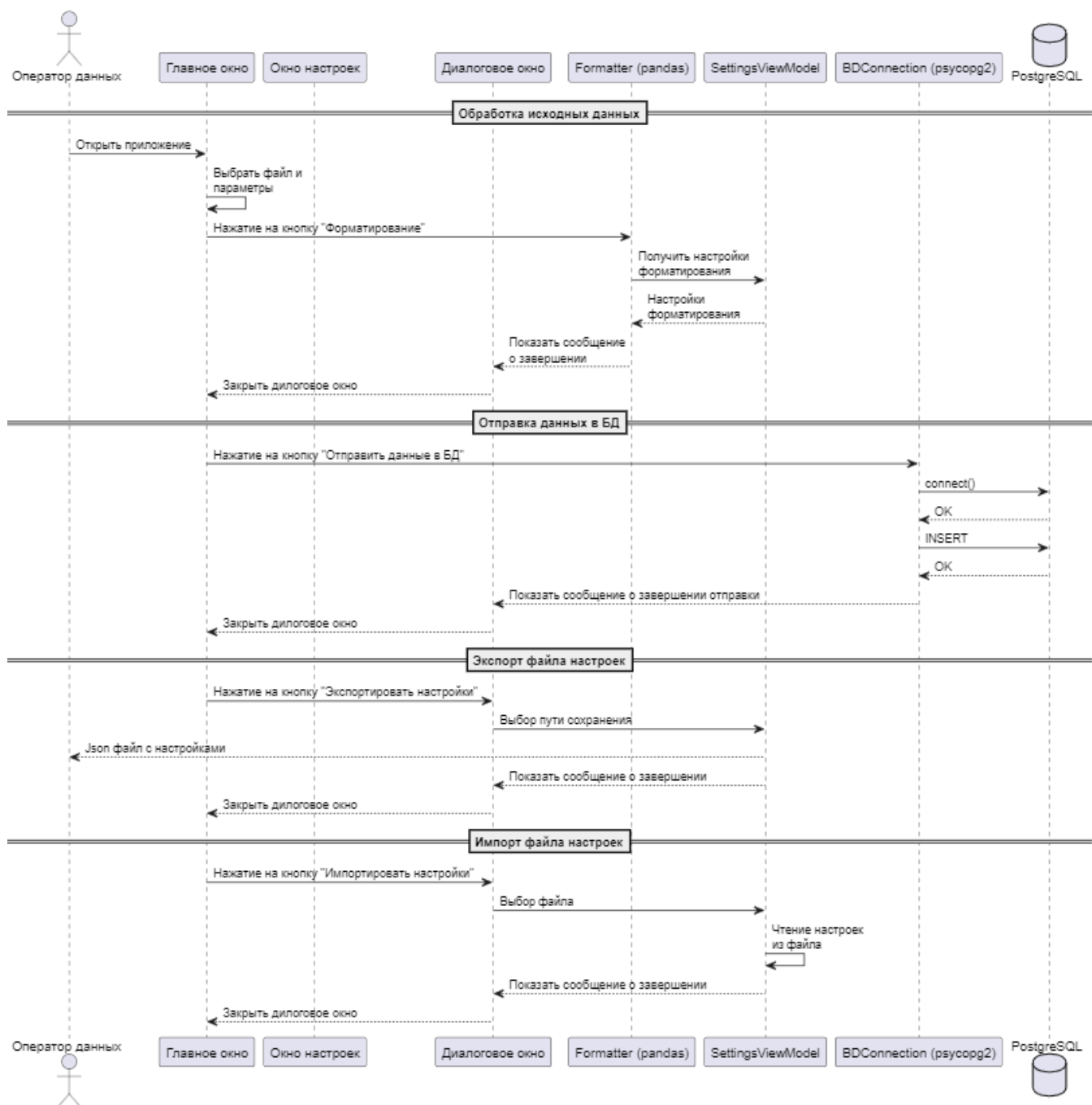


Рисунок 8 - Диаграммы взаимодействия №1

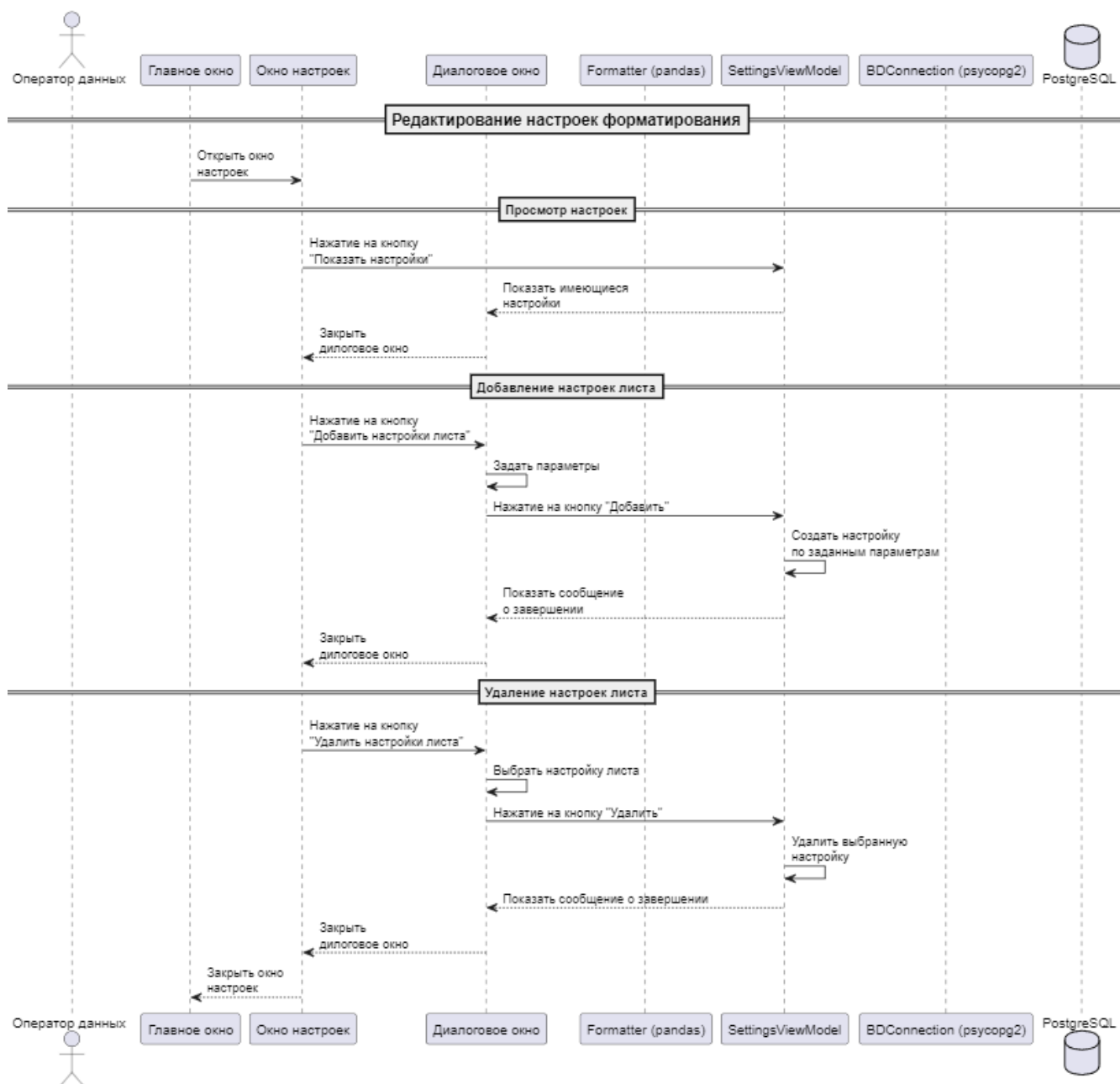


Рисунок 9 - Диаграммы взаимодействия №2

Раздел 3. Реализация проекта

3.1. Контрольный пример работы с программой

Исходный код разработанной системы представлен в приложении А.
Листинг

3.1.1. Пример работы с порталом Visiology.

Для начала работы, в зависимости от пользователя, требуется запустить либо приложение обработки данных, либо портал Visiology.

Рассмотрим работу с порталом Visiology:

На рисунке 10 представлено окно авторизации на портале.

Рисунок 10 - Окно авторизации на портале

Пользователь (Аналитик) может получить доступ к дашбордам визуализирующим данные авторизовавшись на закрытом портале. Для этого ему нужно предварительно получить логин и пароль от службы безопасности.

В отдельных случаях предоставляется временная ссылка для доступа без авторизации.

После авторизации на портале, на экране появятся доступные пользователю дашборды. В нашем случае доступны:

- Кадры. Состав
- Кадры. Замещение, резерв, аттестация, ДПО

На рисунке 11 представлен экран выбора дашбордов.



Рисунок 11 - Экран выбора дашбордов

Открыть дашборд можно кликнув на него. Открыв дашборд, пользователь получает доступ к соответствующим данным. На экране будет доступен выбор листов, на отдельных листах выбор фильтров и выгрузка данных.

На рисунке 12 представлен пример дашборда.

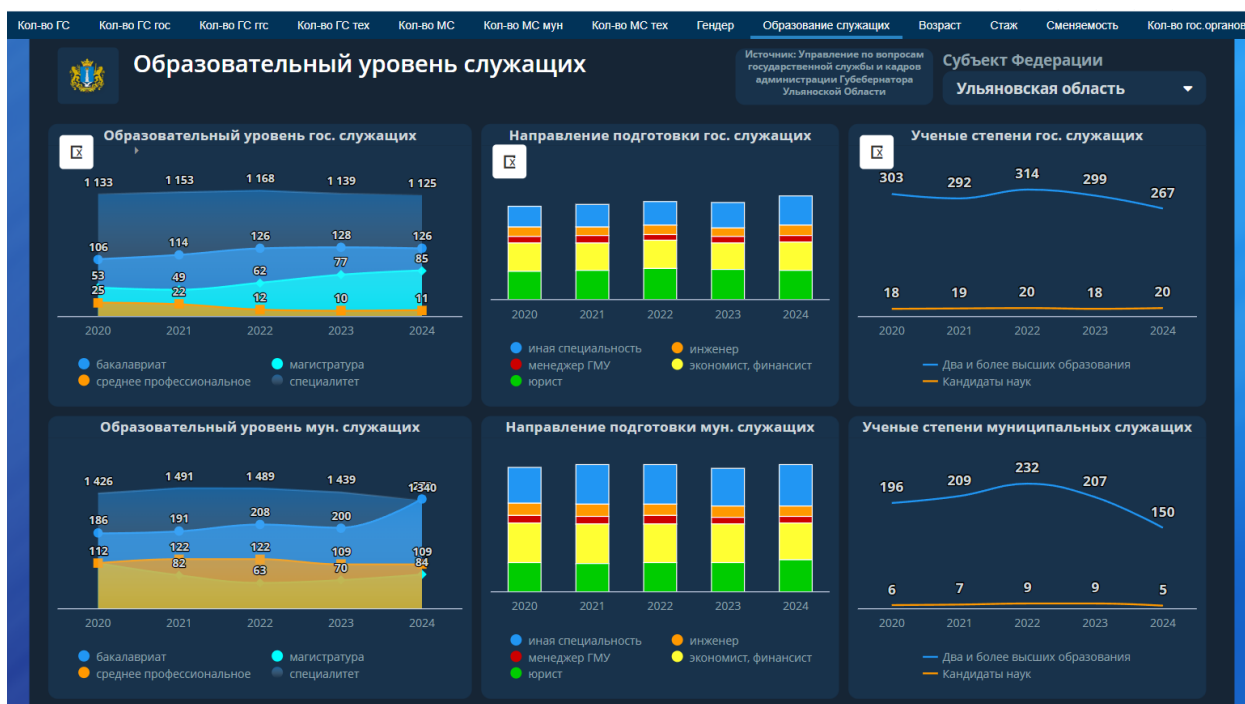


Рисунок 12 - Пример дашборда

3.1.2. Пример работы с приложением обработки данных.

Рассмотрим работу с приложением обработки данных.

Приложение устанавливается на ПК пользователя (оператора данных)

Запустив приложение, пользователь видит главное окно.

На рисунке 13 представлено главное окно приложения.

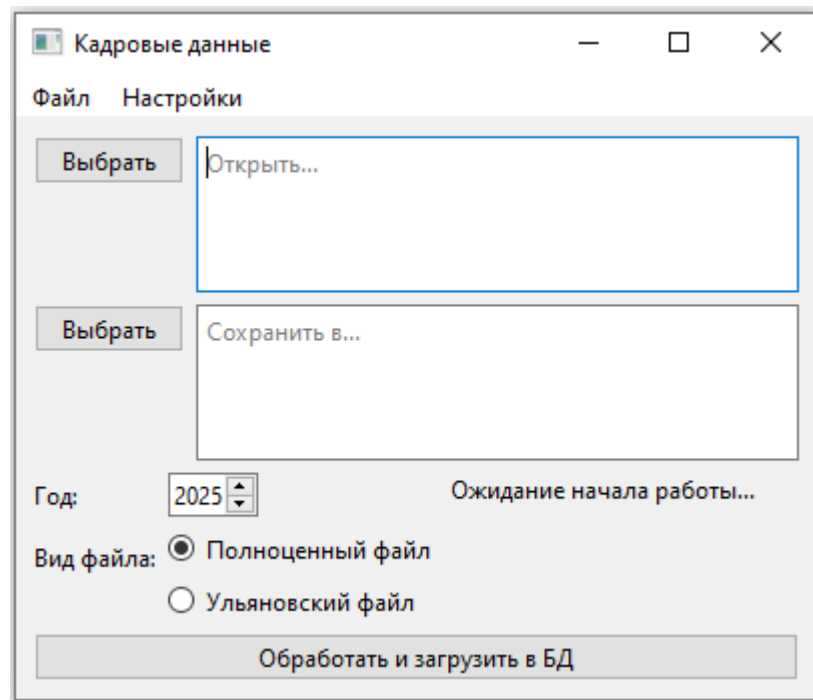


Рисунок 13 - Главное окно приложения

На главном окне пользователь может выбрать файл для обработки, путь для сохранения результата обработки, год и вид файла.

В нижней части экрана находится кнопка начала работы и строка статуса работы приложения.

После указания всех параметров обработки и нажатия на кнопку начала работы, кнопка блокируется, а в строке состояния отображается результат работы.

На рисунке 14 представлен внешний вид приложения в процессе работы.

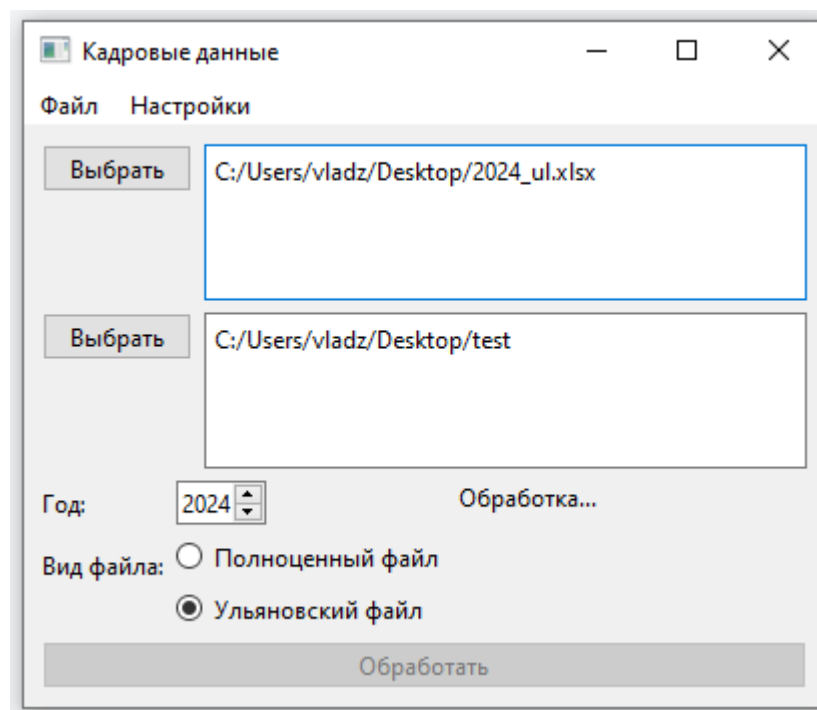


Рисунок 14 - Приложение в работе

На рисунке 15 представлен вид приложения после успешного завершения работы.

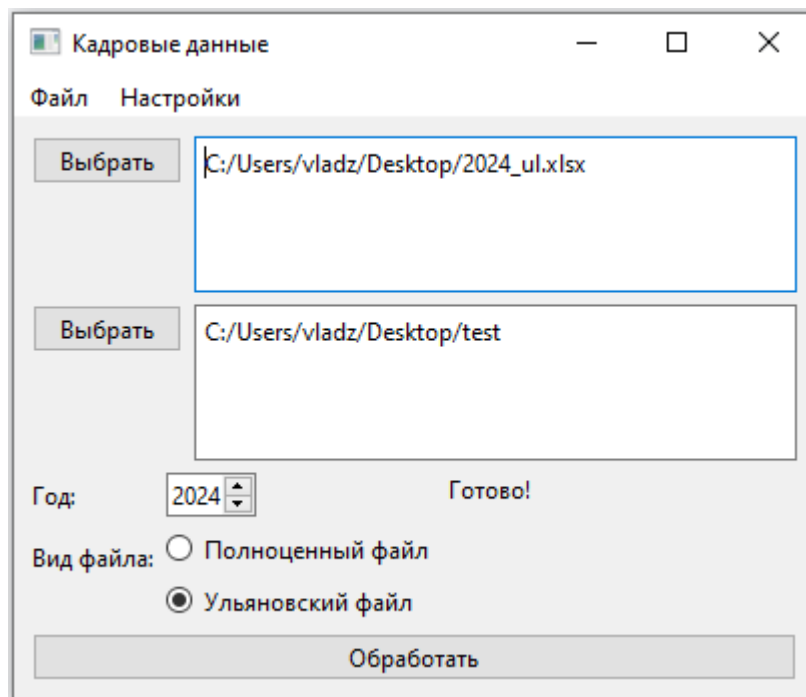


Рисунок 15 - Обработка успешно завершена

Также в верхней части главного окна находится меню с пунктами «Файл» и «Настройки».

На рисунке 16 представлено меню "Файл".

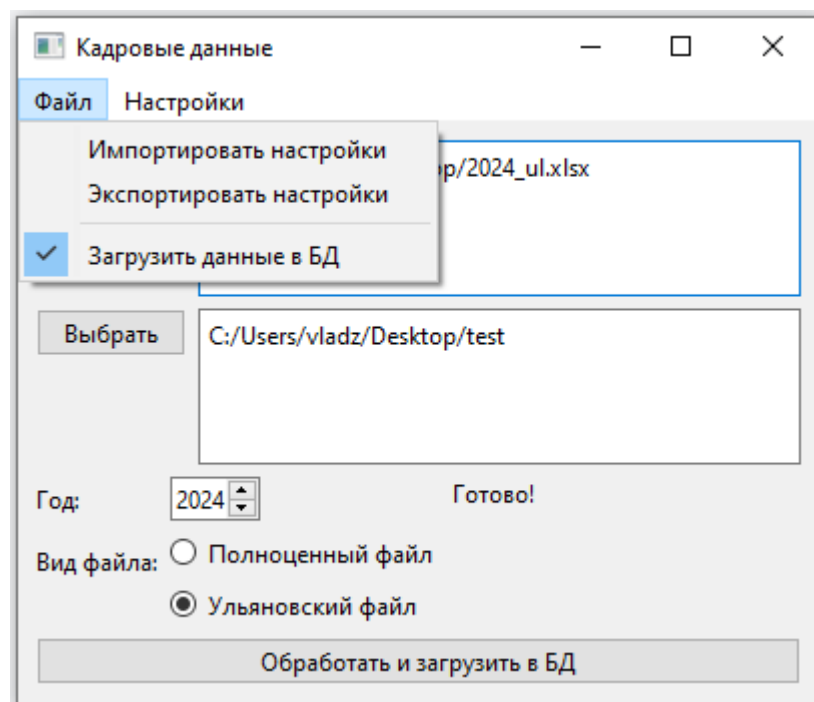


Рисунок 16 - Меню "Файл"

На рисунке 17 представлено меню "Настройки".

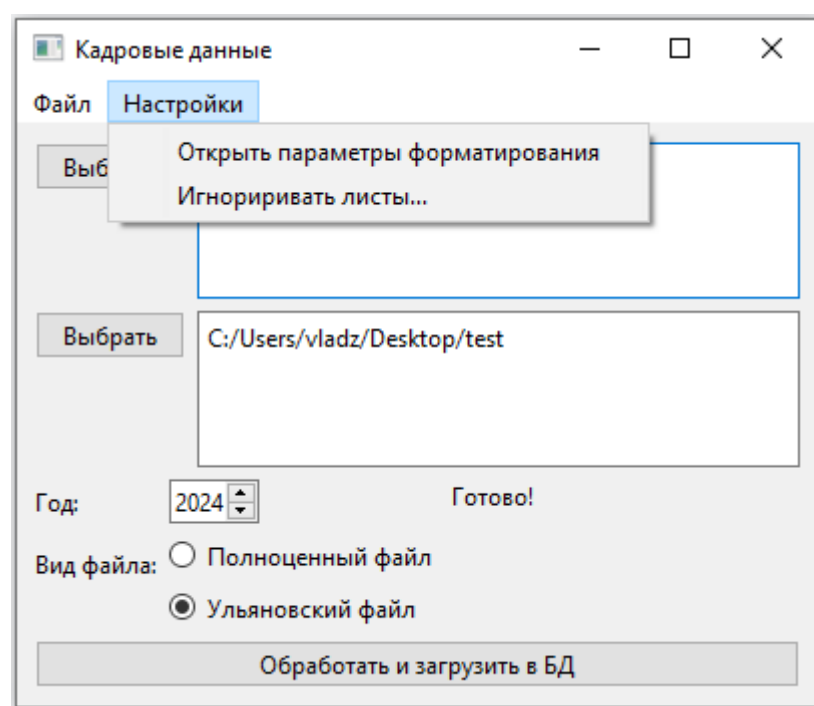


Рисунок 17 - Меню "Настройки"

В меню «Файл» можно экспортировать или импортировать настройки приложения, а также указать, нужно ли сразу загрузить обработанные данные в БД (по умолчанию данные сразу загружаются в БД).

В меню «Настройки» можно открыть окно параметров форматирования и окно выбора игнорирования листов.

На рисунке 18 представлено окно "Игнорировать листы"

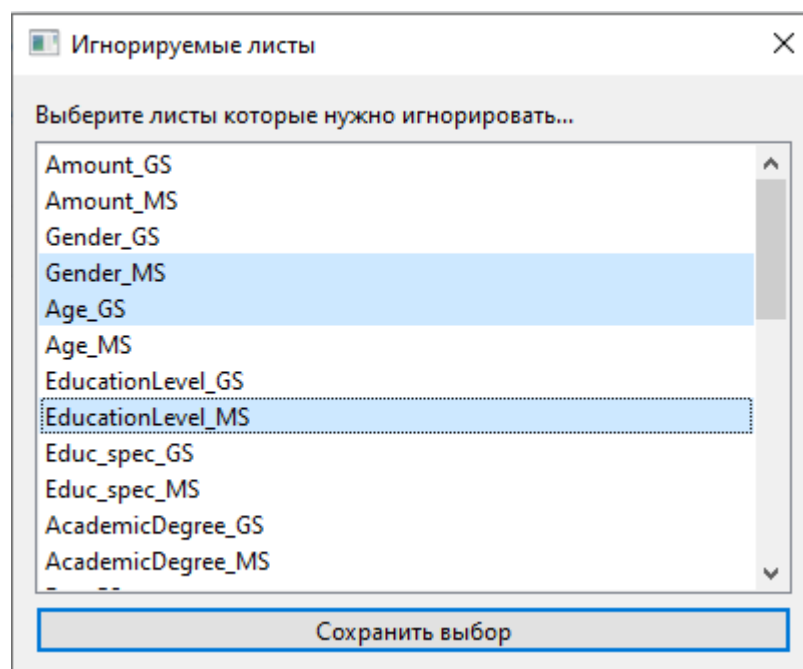


Рисунок 18 - Окно "Игнорировать листы"

В окне «Игнорировать листы» можно выбрать, какие листы выбранной Excel книги нужно пропустить при обработке.

На рисунке 19 представлено окно "Настройки"

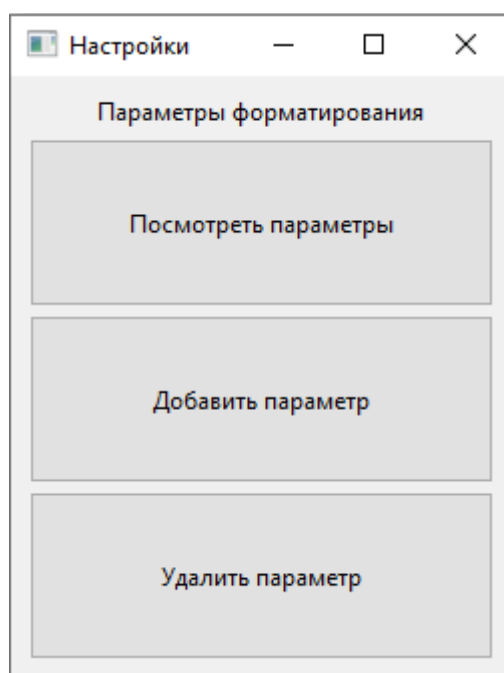


Рисунок 19 - Окно "Настройки"

На окне настроек находятся кнопки три кнопки («Просмотр параметров», «Добавить параметр», «Удалить параметр»), которые открывают соответствующие окна.

На рисунке 20 представлено окно "Параметры"

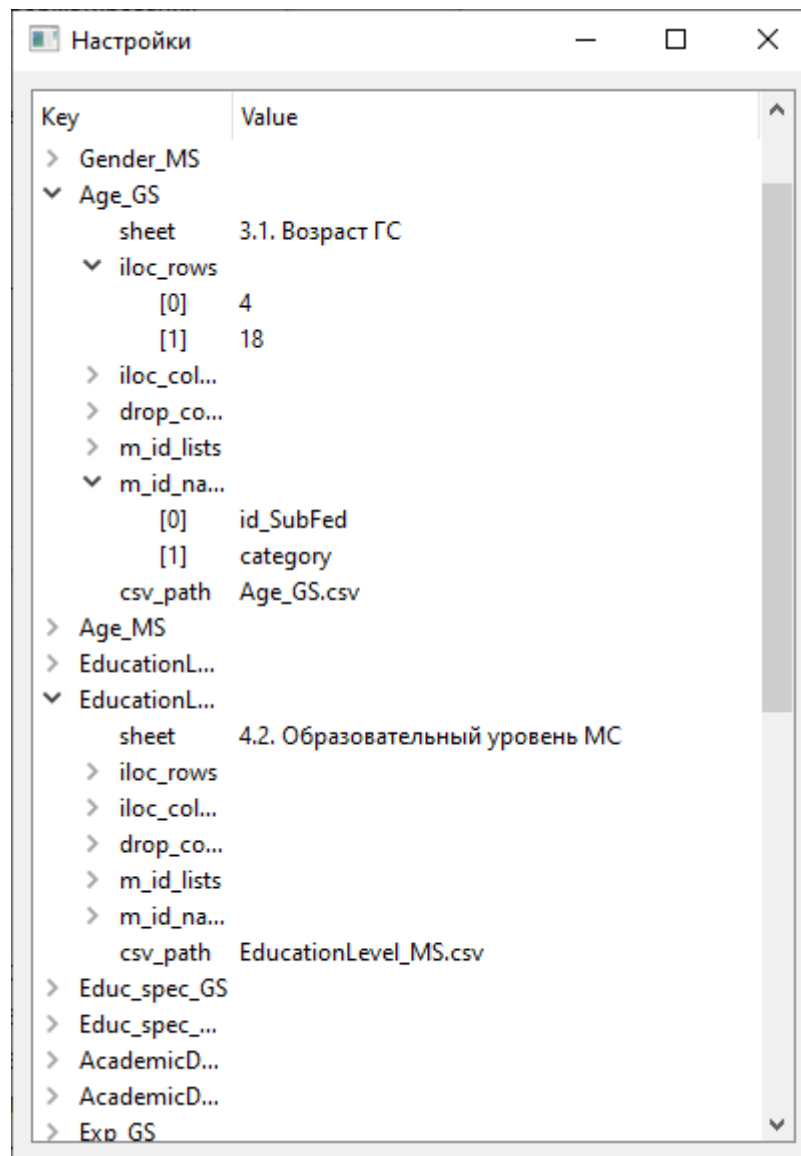


Рисунок 20 - Окно "Параметры"

Открыв окно «Параметры», пользователь может посмотреть заданные в данный момент параметры.

На рисунке 21 представлено окно "Новый лист"

Новый лист

Новый параметр

Название листа (Sheet)

Точное название листа в excel файле

Границы строк (iloc_rows)

Диапазон строк через Enter

Границы столбцов (iloc_columns)

Диапазон столбцов через Enter

Не брать столбцы... (drop_column)

Лишние столбцы через Enter

Списки категорий (m_id_lists)

Уровни мультииндекса, разделяя каждый уровень двойным Enter, каждую категорию начинайте с Enter

Название списков категорий (m_id_nsmes)

Названия списков категорий. Количество названий соответствует количеству уровней мультииндекса

Название финального csv файла (csv_path)

Название csv файла с результатом

Сохранить

Рисунок 21 - Окно "Новый лист"

Открыв окно «Новый лист», пользователь может добавить параметры обработки для нового листа в Excel книге.

На рисунке 22 представлено окно "Удалить лист"

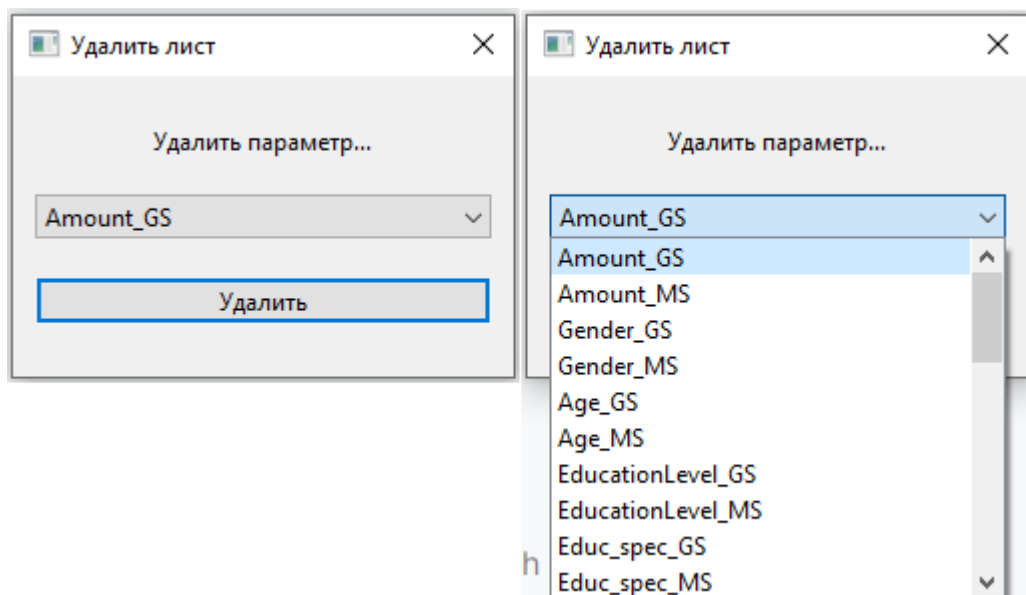


Рисунок 22 - Экран "Удалить лист"

Открыв экран «Удалить лист», пользователь может удалить параметры для выбранного листа.

3.2. План тестирования

В процессе тестирования программного продукта необходимо придерживаться следующего поэтапного плана:

На первом этапе необходимо провести функциональное тестирование, целью которого является проверка функциональных требований к системе, выявление ошибок в каждом отдельном тестируемом модуле (функции).

Функциональное тестирование (functional testing) — это тип тестирования, направленный на проверку правильности работы функционала приложения в соответствии с его функциональными требованиями. Часто данный вид тестирования ассоциируется с методом «чёрного ящика», при котором внимание уделяется входным и выходным данным без анализа внутренней структуры кода.

На втором этапе будет проведено тестирование вывода системных сообщений с целью определения наличия и содержания системных сообщений у функций системы.

На третьем этапе планируется провести тестирование удобства использования, направленное на исследование того, насколько конечному пользователю понятно, как работать с продуктом, а также на то, насколько ему нравится использовать продукт.

На четвёртом этапе планируется проведение тестирования производительности — анализа скорости отклика приложения на внешние воздействия при нагрузках различного характера и интенсивности.

3.3. Протокол тестирования

3.3.1. Функциональное тестирование

Результаты функционального тестирования [13] представлены в таблицах 14-22

Таблица 14 - Результаты функционального тестирования (тест 1)

Идентификатор варианта использования	UC-01
Функция	Загрузка данных в БД
Действие	Оператор данных начинает обработку данных, во время которой данные должны автоматически попасть в БД
Ожидаемый результат	Данные загружены в БД
Предусловие	
Шаги тестирования	1. Открыть приложение обработки данных 2. Заполнить параметры обработки 3. Указать загрузку данных в БД в меню «Файл» 4. Нажать на кнопку начала работы приложения
Постусловие	
Результат	Тест пройден.

Таблица 15 - Результаты функционального тестирования (тест 2)

Идентификатор варианта использования	UC-02
Функция	Импортировать файл настроек
Действие	Оператор данных импортирует полученный ранее файл настроек приложения
Ожидаемый результат	Настройки из файла применяются к приложению
Предусловие	
Шаги тестирования	1. Открыть приложение 2. Открыть меню «Файл» 3. Нажать на кнопку «Импортировать настройки» 4. В появившемся окне выбрать файл
Постусловие	
Результат	Тест пройден.

Таблица 16 - Результаты функционального тестирования (тест 3)

Идентификатор варианта использования	UC-03
--------------------------------------	-------

Функция	Выгрузка данных с дашборда
Действие	Аналитик нажимает кнопку выгрузки данных на одном из виджетов дашборда
Ожидаемый результат	Excel таблица с выбранными данными сохраняется по выбранному пользователем пути
Предусловие	1. Пользователь авторизован на портале 2. Открыт дашборд
Шаги тестирования	1. Открыть Дашборд. 2. Нажать на кнопку выгрузки данных 3. В появившемся окне выбрать путь сохранения файла
Постусловие	
Результат	Тест пройден

Таблица 17 - Результаты функционального тестирования (тест 4)

Идентификатор варианта использования	UC-04
Функция	Настройка игнорирования листов
Действие	Оператор данных в окне «Игнорирование листов» выбирает листы, которые нужно пропустить при обработке
Ожидаемый результат	Листы пропускаются при форматировании
Предусловие	1. Приложение открыто 2. Параметры обработки заполнены
Шаги тестирования	1. Открыть окно игнорирования настроек 2. Выбрать нужные листы 3. Нажать на кнопку начала обработки
Постусловие	
Результат	Тест пройден

Таблица 18 - Результаты функционального тестирования (тест 5)

Идентификатор варианта использования	UC-05
Функция	Выбор фильтров
Действие	Аналитик выбирает доступные на листе дашборда фильтры
Ожидаемый результат	На экран выводятся данные с учетом выбранных фильтров
Предусловие	1. Пользователь авторизован на портале 2. Открыт дашборд
Шаги тестирования	1. Кликнуть на один из фильтров 2. Выбрать один из параметров

Постусловие	
Результат	Тест пройден

Таблица 19 - Результаты функционального тестирования (тест 6)

Идентификатор варианта использования	UC-06
Функция	Добавление параметров листа
Действие	Оператор данных добавляет параметры обработки для нового листа
Ожидаемый результат	Параметр сохраняется и появляется в списке параметров
Предусловие	1. Приложение открыто 2. Открыто окно добавления параметра
Шаги тестирования	1. Заполнить поля параметров 2. Нажать кнопку «Сохранить»
Постусловие	
Результат	Тест пройден

Таблица 20 - Результаты функционального тестирования (тест 7)

Идентификатор варианта использования	UC-07
Функция	Удаление параметров листа
Действие	Оператор данных удаляет параметры обработки листа
Ожидаемый результат	Параметр удаляется из списка параметров
Предусловие	1. Приложение открыто 2. Открыто окно удаления параметра
Шаги тестирования	1. Выбрать параметр 2. Нажать кнопку «Удалить»
Постусловие	
Результат	Тест пройден

Таблица 21 - Результаты функционального тестирования (тест 8)

Идентификатор варианта использования	UC-08
Функция	Просмотр списка параметров
Действие	Оператор данных просматривает список параметров
Ожидаемый результат	Открывается окно показа параметров, заполненное заданными параметрами.
Предусловие	1. Приложение открыто 2. Открыто окно «Настройки»

Шаги тестирования	1. Нажать на кнопку «Посмотреть настройки»
Постусловие	
Результат	Тест пройден

Таблица 22 - Результаты функционального тестирования (тест 9)

Идентификатор варианта использования	UC-09
Функция	Обработка данных без загрузки в БД
Действие	Оператор данных обрабатывает данные, не загружая их в БД
Ожидаемый результат	По указанному пути создаются обработанные CSV файлы
Предусловие	1. Приложение открыто 2. Заданы параметры обработки для тестового файла
Шаги тестирования	1. Нажать кнопку «Выбрать» напротив поля для пути к файлу обработки 2. В открывшемся окне выбрать тестовый файл 3. Нажать кнопку «Выбрать» напротив поля для пути сохранения результата обработки 4. Выбрать тип файла 5. Выбрать год файла 6. Нажать кнопку «Обработать»
Постусловие	
Результат	Тест пройден

3.3.2. Тестирование удобства пользования

Участники тестирования удобства пользования представлены в таблице 23

Результаты тестирования удобства пользования представлены в таблице 24

Таблица 23 - Участники тестирования удобства пользования

ФИО пользователя	Должность
Щанкин Дмитрий Валерьевич	Заместитель начальника управления проектного развития (Региональный проектный офис) администрации Губернатора Ульяновской области – начальник департамента по обеспечению деятельности Ситуационного центра
Глинкин Николай Петрович	Начальник управления проектного развития (Региональный проектный офис) администрации Губернатора Ульяновской области
Воловой Павел Валерьевич	Ведущий экономист экономического отдела отделения Ульяновск Волго-Вятского главного

	управления Центрального Банка Российской Федерации
Кузнецов Александр Валерьевич	Программист
Никулина Надежда Алексеевна	Сотрудник кадрового отдела ГУО

Таблица 24 - Результаты тестирования удобства пользования

Пользователь	Интуитивность	Скорость	Отсутствие ошибок	Общее удовлетворение от работы	Результат
Щанкин Д.В.	9	8	10	9	10
Глинкин Н.П.	10	9	8	9	10
Воловой П.В.	9	10	9	10	7
Кузнецов А.В.	8	9	9	9	8
Никулина Н. А.	9	10	8	9	7

Результаты тестирования: была составлена общая таблица оценок интерфейса пользователями. Средняя оценка равна 8,92. Система соответствует указанным требованиям

3.3.3. Тестирование совместимости

Было произведено тестирование совместимости [19] работы подсистемы анализа в разных браузерах и на разных устройствах.

Результаты тестирования совместимости представлены в таблицах 25 и 26

Таблица 25 - Результаты тестирования совместимости с браузерами

Условие	Chrome	Firefox	Safari	Edge
Загрузка дашборда	Работает корректно	Работает корректно	Работает корректно	Работает корректно
Отображение графиков	Работает корректно	Работает корректно	Работает корректно	Работает корректно
Работа фильтров	Работает корректно	Работает корректно	Работает корректно	Работает корректно
Мобильная версия (Ios/Android)	Есть мелкие артефакты	Есть мелкие артефакты	Работает корректно	Есть мелкие артефакты

Таблица 26 - Результаты тестирования совместимости в разных разрешениях экрана

Условие	1280x720	1920x1080	2560x1440	3840x2160
Масштабирование	Работает корректно	Работает корректно	Мелкие названия листов	Мелкие названия листов
Отображение графиков	Работает корректно	Работает корректно	Работает корректно	Работает корректно

Также было произведено тестирование совместимости приложения обработки данных. Тестирование показало что приложение совместимо с операционной системой Windows версии 7 и младше, что соответствует требованиям.

Результаты тестирования: в самых популярных браузерах и разрешениях экрана – дашборд работает корректно, функциональность не нарушается. Приложение обработки данных корректно работает на протестированных ОС.

3.3.4. Тестирование производительности

Результаты тестирования производительности представлены в таблице 27

Таблица 27 - Результаты тестирования производительности

Метрика	Время
First Contentful Paint	3,9 сек.
Largest Contentful Paint	5,8 сек.
Total Blocking Time	3 620 мс
Speed Index	6,2 сек.
Cumulative Layout Shift	0

Также были в таблице 28 представлены возможные улучшения производительности

Таблица 28 - Возможные улучшения производительности

Предложение	Улучшение
Устранение ресурсов, блокирующих изображение	Потенциальная экономия – 4 190 мс
Удаление неиспользуемого кода JavaScript	Потенциальная экономия – 920 КиБ
Уменьшение размера кода JavaScript	Потенциальная экономия – 41 КиБ
Уменьшение размера кода CSS	Потенциальная экономия – 11 КиБ
Удаление неиспользуемого кода CSS	Потенциальная экономия – 153 КиБ

Результаты тестирования: основные метрики производительности находятся в допустимых интервалах, система соответствует указанным требованиям.

Раздел 4. Оценка эффективности системы

4.1. Расчет трудоемкости работы по разработке системы

Для оценки трудоемкости разработки системы будет использоваться метод UCP (Use Case Points), который оценивает проект, основываясь на сценариях использования:

Определение весовых показателей действующих лиц:

Определение весовых показателей действующих лиц представлено в таблице 29

Таблица 29 - весовые показатели действующих лиц

Тип действующего лица	Весовой коэффициент k_{ai}
Сложный (человек)	3

Общий весовой показатель оценки действующих лиц A вычисляется по формуле:

$$A = \sum n_i * k_{ai} = 1 * 3 = 3, \quad (1)$$

Определение весовых показателей вариантов использования:

Определение весовых показателей вариантов использования представлено в таблице 30

Таблица 30 - весовые показатели вариантов использования

Тип варианта использования	Описание (кол-во классов)	Весовой коэффициент (k_{Bi})
Средний	$4 < x < 7$	10

Общий весовой показатель вариантов использования UC вычисляется по формуле:

$$UC = \sum m_i * k_{Bi} = 1 * 10 = 10 \quad (2)$$

Обобщенный весовой показатель UUCP вычисляется по формуле:

$$UUCP = A + UC = 3 + 10 = 13$$

Определение технической сложности проекта:

Определение технической сложности проекта представлено в таблице 31

Таблица 31 - Техническая сложность проекта

Показатель	Описание	Вес	Значение
T1	Распределенная система	2	0
T2	Высокая производительность (пропускная способность)	1	3
T3	Работа пользователей в режиме online	1	0
T4	Сложная обработка данных	1	5
T5	Повторное использование кода	1	4
T6	Простота установки	0,5	3
T7	Простота использования	0,5	3
T8	Переносимость	2	4
T9	Простота внесения изменений	1	5
T10	Параллелизм	1	2
T11	Специальные требования безопасности	1	0
T12	Непосредственный доступ к системе со стороны внешних пользователей	1	0
T13	Специальные требования к обучению пользователей	1	3

Пояснение для столбца «Значение»:

Каждому показателю T_i присваивается значение в диапазоне от 0 до 5 (0 означает отсутствие значимости показателя для данного проекта, 5 – высокую значимость).

Значение TCF вычисляется по формуле:

$$TCF = 0,6 + (0,01 * \sum(T_i * Вес_i)) = 0,93$$

Определение уровня квалификации разработчиков:

Определение уровня квалификации разработчиков представлено в таблице 32.

Таблица 32 - Уровень квалификации разработчиков

Показатель	Описание	Вес	Пояснение для столбца «Значение»	Значение
F1	Знакомство с технологией	1,5	0 – низкий уровень; 3 – средний; 5 – высокий	2
F2	Опыт разработки	0,5		1
F3	Опыт использования ООП	1		1
F4	Наличие ведущего аналитика	0,5		3
F5	Мотивация	1,5	0-нет мотивации 3-средний уровень 5-высокий уровень	5
F6	Стабильность требований	2	0-высокая нестабильность 3-средняя нестабильность 5-высокая стабильность	3
F7	Частичная занятость	-1	0-нет частичной занятости 3-средний уровень 5-все частично заняты	3
F8	Сложные языки программирования	-1	0-простой ЯП 3-средняя сложность 5-высокая сложность	0

Значение уровня квалификации разработчиков EF вычисляется по следующей формуле:

$$EF = 1,4 + (-0,03 * \Sigma(F_i * \text{Вес}_i)) = 0,905$$

Оценка трудоемкости проекта [18]:

Окончательное значение указателя вариантов использования UCP определяется по формуле:

$$UCP = UUCP * TCF * EF = 13 * 0,93 * 0,905 = 10,94$$

Рассмотрим уточнение трудоемкости за счет учета опыта разработчиков, представленное в таблице 33.

Таблица 33 - Уточнение трудоемкости

Факторы	Число факторов
F(1,2,3,4,5,6)	< 3
F7, F8	> 3
Всего Σ	3

Для системы регистрации получаем 28 чел. часов на одну UCP, таким образом кол-во чел. часов на весь проект равен:

$28 * 10,3974 = 306,36$ часов на весь проект, что составляет ~10 недель при 30-часовой рабочей неделе (5 дней в неделю по 6 часов).

4.2. Анализ затрат на ресурсное обеспечение

4.2.1. Расчет затрат на работу по разработке системы

Затраты на разработку системы рассчитываются по следующей формуле:

$$K_{РПР} = Z_{\text{ФОТР}} + Z_{\text{ОВФ}} + Z_{\text{ЭВМ}} + Z_{\text{СПП}} + Z_{\text{ХОН}} + P_{\text{Н}}, \text{ где}$$

- $Z_{\text{ФОТР}}$ – общий фонд оплаты труда разработчиков ПП;
- $Z_{\text{ОВФ}}$ – начисления на заработную плату разработчиков ПП во внебюджетные фонды;
- $Z_{\text{ЭВМ}}$ – затраты, связанные с эксплуатацией техники;
- $Z_{\text{СПП}}$ – затраты на специальные программные продукты, необходимые для разработки ПП;
- $Z_{\text{ХОН}}$ – затраты на хозяйственно-операционные нужды (бумага, литература, носители информации и т.п.);
- $P_{\text{Н}}$ – накладные расходы ($P_{\text{Н}} = 60\%$ от $Z_{\text{ФОТР}}$).

При разработке программного продукта общее время разработки T_p по расчету трудоемкости составило 3 чел.-мес. Соответственно, машинное время (непосредственная работа с вычислительной и оргтехникой) составляет 3 месяца.

Фонд оплаты труда за время работы над программным продуктом:

$$З_{\text{ФОР}} = \left(\sum_{j=1}^m o_{pj} \cdot d_{\text{РПР}j} \right) \cdot T_p \cdot (1 + k_d), \quad (3)$$

где O_{pj} – оклад j -го разработчика;

$d_{\text{РПР}j}$ – доля j -го разработчика в общем времени работы T_p над проектом в месяцах;

k_d – коэффициент дополнительной зарплаты при наличии.

Проект выполняется одним человеком в рамках одной должности – разработчика с заработной платой в 26 000 руб.

Тогда $З_{\text{ФОР}} = (26\,000 \cdot 1) \cdot 3 \cdot 1 = 78\,000$ руб.

Сумма начислений на заработную плату во внебюджетные фонды рассчитывается:

$$З_{\text{ОВФ}} = З_{\text{ФОР}} \cdot k_{\text{ОВФ}}, \quad (4)$$

где $k_{\text{ОВФ}}$ – коэффициент отчислений во внебюджетные фонды.

Получается, $З_{\text{ОВФ}} = 78\,000 \cdot 0,302 = 23\,556$ руб.

Затраты, связанные с использованием вычислительной и оргтехники:

$$З_{\text{ЭВМ}} = T_{\text{МРПР}} \cdot k_r \cdot n \cdot C_{\text{М-ч}}, \quad (5)$$

где k_r – коэффициент готовности ЭВМ;

n – количество единиц техники;

$C_{\text{М-ч}}$ – себестоимость машиночаса в рублях;

$T_{\text{МРПР}}$ – машинное время работы над программным продуктом.

Разработка ведется с использованием собственной вычислительной техники, следовательно, затраты связанные с эксплуатацией техники равны 0.

Для разработки ПП необходим специальный программный продукт «Платформа Visiology». На предприятии он уже есть, поэтому $Z_{СПП} = 0$ руб.

Затраты на хозяйственно-организационные нужды рассчитываются по формуле:

$$Z_{ХОН} = \sum_{j=1}^n C_j \cdot K_j, \quad (6)$$

Поскольку в ходе работы над проектом бумажные носители не использовались, затраты на хозяйственно-организационные нужды равны 0.

Накладные расходы рассчитываются как $P_H = Z_{ФОТР} \cdot k_{НР}$, где $k_{НР}$ – коэффициент накладных расходов, определяется по данным организации

Накладные расходы составили $78\,000 \cdot 0,2 = 15\,600$ руб.

Таким образом, общие затраты на разработку программного продукта составят:

$$K_{РПР} = 78\,000 + 23\,556 + 0 + 0 + 0 + 15\,600 = 117\,156$$

4.2.2. Расчет стоимости эксплуатации оборудования

Рассчитывается в зависимости от используемого оборудования на предприятии внедрения разрабатываемого программного продукта.

Согласно информации от заказчика, эксплуатация оборудования не требует каких-либо затрат.

4.3. Анализ качественных и количественных факторов воздействия проекта на бизнес-архитектуру организации

4.3.1. Расчет экономических результатов от внедрения

Внедрение приложения позволяет сократить время, затрачиваемое на анализирование данных и подготовку отчётности.

На данный момент процесс подготовки ежегодной отчетности занимает от трех до 4 недель рабочего времени (около 100 часов), без учёта этапа сбора данных. С использованием ПП этот процесс занимает не более 5 часов.

Суммарная экономия времени в год составляет 95 ч. Или $95 / 12 = \sim 8$ часов в месяц

Средняя заработная плата в компании составляет 26 000 рублей в месяц. В общей сложности накладные расходы, например, в среднем составят примерно 0,7 от величины оклада. Таким образом, содержание одного сотрудника обходится компании в среднем 44 200 рублей в месяц.

Стоимость одного рабочего часа рассчитывается $\text{Стоимость}_\text{ч} = \frac{\text{СтоимостьСодержанияСотрудника}}{K_{\text{рабоч. дней мес.}} * P_{\text{рабоч. дня}}}$, где $P_{\text{рабоч. дня}}$ – продолжительность рабочего дня.

$$\text{Получается, } \text{Стоимость}_\text{ч} = \frac{44200}{22 * 6} \approx 334 \text{ руб./ч.}$$

Экономия денег за месяц рассчитывается как:

$$\text{Э}_{\text{мес.}} = \text{Э}_{\text{ч. в мес.}} * \text{Стоимость}_\text{ч}, \quad (7)$$

Получается, что $\text{Э}_{\text{мес.}} = 8 * 334 = 2672$ руб.

Экономия организации составит **2672 рублей в месяц** из общего фонда заработной платы.

4.3.2. Расчет сроков окупаемости проекта

Срок окупаемости проекта рассчитывается по формуле:

$$\text{СрокОкупаемости} = \frac{\text{Затраты на разработку}}{\text{ЭкономияВМесяц} - \text{РасходыВМесяц}} = \frac{117156}{2672 - 0} = 44 \text{ мес.}$$

Заключение

В результате выполнения выпускной квалификационной работы была проанализирована предметная область, сформированы цели и задачи. Была спроектирована и реализована система, состоящая подсистема анализа и подсистема хранения кадровых данных СЦ ГУО, были спроектированы: диаграмма прецедентов, ER-диаграмма, диаграмма классов, диаграмма компонентов, диаграмма развертывания, диаграммы взаимодействия.

Разработанная система отвечает всем требованиям, внедрена и используется заказчиком.

Список литературы

1. ER-модель [Электронный ресурс] - Электрон. дан. - Режим доступа: <https://ru.wikipedia.org/wiki/ER-модель>. - Загл. с экрана
2. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил.
3. Qt Designer Manual [Электронный ресурс] // Qt Documentation. — URL: <https://doc.qt.io/qt-6/qtdesigner-manual.html> (дата обращения: 04.06.2025).
4. Балашов А.И., Рогова Е.М., Тихонова М.В., Ткаченко Е.А. Управление проектами: учебник для бакалавров / под. ред. Е.М. Роговой. — М.: Издательство Юрайт, 2013 — 383 с. — Серия: Бакалавр. Базовый курс.
5. Белик, А. Г. Проектирование и архитектура программных систем: учебное пособие / А. Г. Белик, В. Н. Цыганенко – Омск: ОмГТУ, 2016 – 96с.
6. Бибо Бер , Кац Иегуда jQuery. Подробное руководство по продвинутому JavaScript; Символ-плюс - М., 2017. - 624 с.
7. Галиаскаров, Э. Г. Анализ и проектирование систем с использованием UML: учебное пособие для вузов / Э. Г. Галиаскаров, А. С. Воробьев. - Москва : Издательство Юрайт, 2022. - 125 с.
8. Документация по Visiology [Электронный ресурс] // Visiology. — URL: <https://visiology-doc.atlassian.net/wiki/spaces/doc/overview> (дата обращения: 01.03.2025).
9. Леоненков А. В. Нотация и семантика языка UML. — Национальный Открытый Университет "ИНТУИТ", 2016. — 205 с.
10. Леоненков А. В. Самоучитель UML 2. — СПб.: БХВ-Петербург, 2007. — 576 с.
11. Миронов, В. Профессия "бизнес-аналитик" : краткое пособие для начинающих / В. Миронов. — 3-е изд., испр. и доп.. — Москва : Олимп-Бизнес, 2023. — 217 с.

12. Моисеев А.Н., Литовченко М.И. Основы языка UML : учеб. пособие. – Томск : Издательство Томского государственного университета, 2023. – 96 с.
13. Морозова, Ю. В. Тестирование программного обеспечения : учебное пособие / Ю. В. Морозова. – Томск : Эль Контент, 2019. – 120 с.
14. Невзорова О.А., Миннегалиева Ч.Б. Основы UML / О.А. Невзорова, Ч.Б. Миннегалиева. – Казань: Казан. ун-т, 2021. – 43 с.
15. Новиков Д.А. Теория управления организационными системами. — М.: МПСИ, 2005 - 584 с
16. Основы программирования на языке Python : учебное пособие / С. К. Буйначев, Н. Ю. Боклаг. – Екатеринбург : Изд-во Урал. ун-та, 2014. – 91 с.
17. Проектный практикум. Управление проектом в ProjectLibre: учебно-методическое пособие / Сост.: А.Ю. Сапаров, Ижевск, 2020. 48 с
18. Самочадин, А. В. Оценка трудоемкости разработки программного обеспечения : учебное пособие / А. В. Самочадин, Т. Н. Самочадина. — Санкт-Петербург : Санкт-Петербургский политехнический университет Петра Великого, 2023. — 87 с.
19. Тестирование программного обеспечения. Базовый курс обеспечения : учебное пособие / С. С. Куликов . – [Электронный ресурс] – Режим доступа: <https://its.1c.ru/db/v8std.> - Загл. с экрана
20. Фаулер М. UML. Основы, 3-е издание. – Пер. с англ. – СПб: Символ-Плюс, 2004. – 192 с.

Приложение А

Листинг кода

```
Файл connection.py
from sqlalchemy import create_engine, text
from config_project import Const

engine = create_engine(url=Const.DATABASE_URL_psycpg())

with engine.connect() as conn:
    res = conn.execute(text("select version()"))
    print(f'{res.all()}')
```

Файл model_settings.py

```
from pydantic import BaseModel

class Setting(BaseModel):
    sheet: str
    iloc_rows: list[int]
    iloc_columns: list[int]
    drop_column: list[int]
    m_id_lists: list[list[str]]
    m_id_names: list[str]
    csv_path: str
```

Файл sheet_formatting.py

```
import os

import pandas as pd
from pandas import DataFrame, MultiIndex, read_excel

from sheet_format.model_settings import Setting
from database.connection import engine

class BaseFormatter:
    ID_SUB_FED = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,
13]
    ULSK = [13]

    @staticmethod
    def save_to_csv(dataframe: DataFrame, file_path: str,
separator: str) -> None:
        """Создаем нужные папки если их нет и
конвертируем DataFrame в CSV"""

        folder = os.path.dirname(file_path)
        if not os.path.exists(folder):
            os.makedirs(folder)
        dataframe.to_csv(file_path, sep=separator)

    @staticmethod
    def data_frame_from_excel(path: str, settings:
Setting) -> DataFrame:
        """Получаем DataFrame из файла"""
        df = read_excel(path, sheet_name=settings.sheet,
header=None)
        df = df.iloc[settings.iloc_rows[0]:settings.iloc_rows[1],
settings.iloc_columns[0]:settings.iloc_columns[1]] # Обрезаем
лишнее
        df = df.drop(settings.drop_column, axis=1)
        return df

    def start_format(self, sheet_settings: [str, Setting],
file_path: str, file_year: int, save_path: str,
ignore_sheet: set, insert_to_base: bool,
excel_to_csv=None):
        file_name = file_path.split("/")[-1]

        print(f"\nОткрываем файл ' + file_name)
        for key, settings in sheet_settings.items():
            if key in ignore_sheet:
                continue
            df = excel_to_csv(file_path, file_year, settings)
            self.save_to_csv(df, 'CSVs/' + settings.csv_path,
';')
            self.save_to_csv(df, save_path + '/result/' +
settings.csv_path, ';')
            if insert_to_base:
                insert_df = pd.read_csv(f"CSVs/{settings.csv_path}", sep=';')
                insert_df.to_sql(key, engine,
if_exists='append', index=False)

class FullFormatter(BaseFormatter):
    def excel_to_csv(self, path: str, year: int, settings:
Setting) -> DataFrame:
```

```

        """Функции обработки для полноценных
        файлов"""

        print('Обработка ' + settings.sheet + '...')
        df = self.data_frame_from_excel(path, settings)
        df = df.stack().reset_index(drop=True).to_frame()
# Складываем таблицу в одну большую строку
        m_id = MultiIndex.from_product([self.ID_SUB_FED]
        settings.m_id_lists, names=settings.m_id_names)
        df = df.set_axis(m_id, axis=0) # Присваиваем
        мультииндекс
        df.insert(0, 'year', str(year) + '-01-01') # Добавляем
        колонку "год"
        df = df.rename(columns={0: 'amount'})
        return df

    def start_format(self, sheet_settings: [str, Setting],
        file_path: str, file_year: int, save_path: str,
        ignore_sheet: set, insert_to_base: bool):
        super().start_format(sheet_settings, file_path,
        file_year, save_path, ignore_sheet,
        insert_to_base, self.excel_to_csv)

class UlskFormater(BaseFormater):
    def excel_to_csv(self, path: str, year: int, settings:
        Setting) -> DataFrame:
        """Функции обработки для ульяновских
        файлов"""

        print('Обработка ' + settings.sheet + '...')
        df = self.data_frame_from_excel(path, settings)
        df = df.dropna(how='any')
        if df.size == 0:
            print('Ошибка ' + settings.sheet)
            return df
        df = df.stack().reset_index(drop=True).to_frame()
# Складываем таблицу в одну большую строку
        m_id = MultiIndex.from_product([self.ULSK] +
        settings.m_id_lists, names=settings.m_id_names)
        df = df.set_axis(m_id, axis=0) # Присваиваем
        мультииндекс
        df.insert(0, 'year', str(year) + '-01-01') # Добавляем
        колонку "год"
        df = df.rename(columns={0: 'amount'})
        return df

    def start_format(self, sheet_settings: [str, Setting],
        file_path: str, file_year: int, save_path: str,
        ignore_sheet: set, insert_to_base: bool):
        super().start_format(sheet_settings, file_path,
        file_year, save_path, ignore_sheet,
        insert_to_base, self.excel_to_csv)

```

```

Файл sheet_settings.py
import json

from config_project import Const
from sheet_format.model_settings import Setting

def update_or_reset_settings(settings: dict[str, Setting]) -
> None:
    print('Обновление файла настроек...')
    for key, value in settings.items():
        settings[key] = value.model_dump()
    json_string = json.dumps(settings,
        ensure_ascii=False, indent=4)
    with open(Const.SETTINGS_FILE, "w",
        encoding="utf-8") as file:
        file.write(json_string)
    print('SUCCESS')

def get_json_string() -> str:
    try:
        with open(Const.SETTINGS_FILE, "r",
            encoding="utf-8") as file:
            json_string = json.load(file)
            print("Настройки форматирования
            получены!")
            return json.dumps(json_string, ensure_ascii=False,
            indent=4)
    except FileNotFoundError:
        return ""

def get_settings() -> dict[str, Setting]:
    try:
        with open(Const.SETTINGS_FILE, "r",
            encoding="utf-8") as file:
            dict_data: dict = json.load(file)
            print("Настройки форматирования
            получены!")
            for key, value in dict_data.items():
                dict_data[key] = Setting(**value)
            return dict_data
    except FileNotFoundError:
        return {}

Файл WorkerDoFormat.py
from PySide6.QtCore import QThread, Signal

from sheet_format.model_settings import Setting
from sheet_format.sheet_formatting import
BaseFormater

```

```

class WorkerDoFormat(QThread):
    signal = Signal(str, bool)

    def __init__(self, sheet_settings: [str, Setting], mode:
BaseFormater, file_path: str, file_year: int,
        save_path: str, ignore_sheet: set, insert_to_db:
bool):
        super().__init__()
        self.sheet_settings = sheet_settings
        self.mode = mode
        self.file_path = file_path
        self.file_year = file_year
        self.save_path = save_path
        self.ignore_sheet = ignore_sheet
        self.insert_to_db = insert_to_db

    def run(self):
        self.signal.emit("Обработка...", True)
        try:
            self.mode.start_format(self.sheet_settings,
self.file_path, self.file_year,
                self.save_path, self.ignore_sheet,
self.insert_to_db)
            self.signal.emit("Готово!", False)
        except Exception as e:
            self.signal.emit("Форматирование не
удалось!", False)
            print(e)

Файл ui_main.py

from PySide6.QtCore import (QCoreApplication,
QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QAction, QBrush, QColor,
QConicalGradient,
    QCursor, QFont, QFontDatabase, QGradient,
    QIcon, QImage, QKeySequence, QLinearGradient,
    QPainter, QPalette, QPixmap, QRadialGradient,
    QTransform)
from PySide6.QtWidgets import (QApplication,
QFormLayout, QHBoxLayout, QLabel,
    QMainWindow, QMenu, QMenuBar, QPushButton,
    QRadioButton, QSizePolicy, QSpinBox, QTextEdit,
    QVBoxLayout, QWidget)

class Ui_MainWindow(object):
    def setupUi(self, MainWindow):
        if not MainWindow.setObjectName():

            MainWindow.setObjectName(u"MainWindow")
            MainWindow.resize(400, 300)
            MainWindow.setMinimumSize(QSize(400, 300))
            self.actionOpenSheetSettingsWindow =
QAction(MainWindow)
            self.actionOpenSheetSettingsWindow.setObjectName(u"actionOpe
nSheetSettingsWindow")
            self.actionSettingsListIgnore =
QAction(MainWindow)
            self.actionSettingsListIgnore.setObjectName(u"actionSettingsListI
gnore")
            self.actionImport = QAction(MainWindow)
            self.actionImport.setObjectName(u"actionImport")
            self.actionExport = QAction(MainWindow)
            self.actionExport.setObjectName(u"actionExport")
            self.actionInsertData = QAction(MainWindow)
            self.actionInsertData.setObjectName(u"actionInsertData")
            self.actionInsertData.setCheckable(True)
            self.actionInsertData.setChecked(True)
            self.centralwidget = QWidget(MainWindow)
            self.centralwidget.setObjectName(u"centralwidget")
            self.verticalLayout_3 =
QVBoxLayout(self.centralwidget)
            self.verticalLayout_3.setObjectName(u"verticalLayout_3")
            self.formLayout = QFormLayout()
            self.formLayout.setObjectName(u"formLayout")
            self.textEditSelectedFilePath =
QTextEdit(self.centralwidget)
            self.textEditSelectedFilePath.setObjectName(u"textEditSelectedFil
ePath")
            sizePolicy =
QSizePolicy(QSizePolicy.Policy.Expanding,
QSizePolicy.Policy.Expanding)
            sizePolicy.setHorizontalStretch(0)
            sizePolicy.setVerticalStretch(0)
            sizePolicy.setHeightForWidth(self.textEditSelectedFilePath.sizePo
licy().hasHeightForWidth())
            self.textEditSelectedFilePath.setSizePolicy(sizePolicy)
            self.formLayout.addWidget(
QFormLayout.FieldRole, self.textEditSelectedFilePath)
            self.textEditResultPath =
QTextEdit(self.centralwidget)

```

```

self.textEditResultPath.setObjectName(u"textEditResultPath")

sizePolicy.setHeightForWidth(self.textEditResultPath.sizePolicy().
hasHeightForWidth())
    self.textEditResultPath.setSizePolicy(sizePolicy)

    self.formLayout.addWidget(1,
QFormLayout.FieldRole, self.textEditResultPath)

    self.btnOpenFile =
QPushButton(self.centralwidget)
    self.btnOpenFile.setObjectName(u"btnOpenFile")
    sizePolicy1 =
QSizePolicy(QSizePolicy.Policy.Maximum,
QSizePolicy.Policy.Maximum)
    sizePolicy1.setHorizontalStretch(0)
    sizePolicy1.setVerticalStretch(0)

sizePolicy1.setHeightForWidth(self.btnOpenFile.sizePolicy().hasH
eightForWidth())
    self.btnOpenFile.setSizePolicy(sizePolicy1)

    self.formLayout.addWidget(0,
QFormLayout.LabelRole, self.btnOpenFile)

    self.btnSelectResultPath =
QPushButton(self.centralwidget)

self.btnSelectResultPath.setObjectName(u"btnSelectResultPath")

sizePolicy1.setHeightForWidth(self.btnSelectResultPath.sizePolic
y().hasHeightForWidth())

self.btnSelectResultPath.setSizePolicy(sizePolicy1)

    self.formLayout.addWidget(1,
QFormLayout.LabelRole, self.btnSelectResultPath)

    self.verticalLayout_3.addLayout(self.formLayout)

    self.horizontalLayout = QHBoxLayout()

self.horizontalLayout.setObjectName(u"horizontalLayout")
    self.formLayoutSettings = QFormLayout()

self.formLayoutSettings.setObjectName(u"formLayoutSettings")
    self.labelYear = QLabel(self.centralwidget)
    self.labelYear.setObjectName(u"labelYear")

sizePolicy2 =
QSizePolicy(QSizePolicy.Policy.Maximum,
QSizePolicy.Policy.Preferred)
    sizePolicy2.setHorizontalStretch(0)
    sizePolicy2.setVerticalStretch(0)

sizePolicy2.setHeightForWidth(self.labelYear.sizePolicy().hasHei
ghtForWidth())
    self.labelYear.setSizePolicy(sizePolicy2)

    self.formLayoutSettings.addWidget(0,
QFormLayout.LabelRole, self.labelYear)

    self.spinBoxFileYear =
QSpinBox(self.centralwidget)

self.spinBoxFileYear.setObjectName(u"spinBoxFileYear")
    sizePolicy3 =
QSizePolicy(QSizePolicy.Policy.Maximum,
QSizePolicy.Policy.Fixed)
    sizePolicy3.setHorizontalStretch(0)
    sizePolicy3.setVerticalStretch(0)

sizePolicy3.setHeightForWidth(self.spinBoxFileYear.sizePolicy().
hasHeightForWidth())
    self.spinBoxFileYear.setSizePolicy(sizePolicy3)
    self.spinBoxFileYear.setMinimum(2000)
    self.spinBoxFileYear.setMaximum(3000)
    self.spinBoxFileYear.setValue(2025)

    self.formLayoutSettings.addWidget(0,
QFormLayout.FieldRole, self.spinBoxFileYear)

    self.labelFileType = QLabel(self.centralwidget)

self.labelFileType.setObjectName(u"labelFileType")

sizePolicy2.setHeightForWidth(self.labelFileType.sizePolicy().has
HeightForWidth())
    self.labelFileType.setSizePolicy(sizePolicy2)

    self.formLayoutSettings.addWidget(1,
QFormLayout.LabelRole, self.labelFileType)

    self.verticalLayout = QVBoxLayout()

self.verticalLayout.setObjectName(u"verticalLayout")
    self.radioBtnFullFile =
QRadioButton(self.centralwidget)

self.radioBtnFullFile.setObjectName(u"radioBtnFullFile")
    self.radioBtnFullFile.setChecked(True)

```



```

        u"\u0418\u043c\u043f\u043e\u0440\u0442\u0438\u0440\u043e\u0432\u0430\u0442\u044c"
        \u043d\u0430\u0441\u0442\u0440\u043e\u0439\u043a\u0438",
        None))

    self.actionExport.setText(QCoreApplication.translate("Main Window",
        u"\u042d\u043a\u0441\u043f\u043e\u0440\u0442\u0438\u0440\u043e\u0432\u0430\u0442\u044c"
        \u043d\u0430\u0441\u0442\u0440\u043e\u0439\u043a\u0438",
        None))

    self.actionInsertData.setText(QCoreApplication.translate("Main Window",
        u"\u0417\u0430\u0433\u0440\u0443\u0443\u0437\u0438\u0442\u044c"
        \u0434\u0430\u043d\u044b\u044b \u0432 \u0432\u0438\u0434\u0435 \u0441\u0432\u0435\u0442\u0430",
        None))

    self.textEditSelectedFilePath.setPlaceholderText(QCoreApplication.translate("Main Window",
        u"\u041e\u0442\u043a\u0440\u044b\u0442\u044c...", None))

    self.textEditResultPath.setPlaceholderText(QCoreApplication.translate("Main Window",
        u"\u0421\u043e\u0445\u0445\u0440\u0430\u043d\u0438\u0442\u044c"
        \u0432\u0432\u043e\u0434\u0435 \u0441\u0432\u0435\u0442\u0430...", None))

    self.btnOpenFile.setText(QCoreApplication.translate("Main Window",
        u"\u0412\u044b\u0431\u0440\u0430\u0442\u044c", None))

    self.btnSelectResultPath.setText(QCoreApplication.translate("Main Window",
        u"\u0412\u044b\u0431\u0440\u0430\u0442\u044c", None))

    self.labelYear.setText(QCoreApplication.translate("Main Window",
        u"\u0413\u043e\u0434:", None))

    self.labelFileType.setText(QCoreApplication.translate("Main Window",
        u"\u0412\u0438\u0434\u0435 \u0442\u044b\u0430\u0439\u043b\u0430:",
        None))

    self.radioBtnFullFile.setText(QCoreApplication.translate("Main Window",
        u"\u041f\u043e\u043b\u043d\u043e\u0446\u0435\u043d\u043d\u043e \u044b\u0439\u0440\u0430\u0439\u0438",
        None))

    self.radioBtnULFile.setText(QCoreApplication.translate("Main Window",
        u"\u0423\u043b\u0446\u044f\u043d\u043e\u0432\u0441\u0442\u0441\u0430 \u044b\u0439\u0438",
        None))

    self.labelStatus.setText(QCoreApplication.translate("Main Window

```

```

", u"\u041e\u0436\u0438\u0434\u0430\u043d\u0438\u0432\u0430\u0447\u0430\u043b\u0430"
\u0440\u0430\u0431\u043e\u0442\u044b...", None))

self.btnDoFormatToCSV.setText(QCoreApplication.translate("Main Window",
    u"\u041e\u0431\u0440\u0430\u0438\u043e\u0442\u0430\u0442\u044c"
    \u0438 \u0437\u0430\u0433\u0440\u0443\u0443\u0437\u0438\u0442\u044c"
    \u0432 \u0441\u0432\u0435\u0442\u0430", None))

self.menu_settings.setTitle(QCoreApplication.translate("Main Window",
    u"\u0412\u0430\u0441\u0442\u0440\u043e\u0439\u043a\u0438",
    None))

self.menu_file.setTitle(QCoreApplication.translate("Main Window",
    u"\u0424\u0430\u0439\u043b", None))

    # retranslateUi
    Файл ui_settings.py

    from PySide6.QtCore import (QCoreApplication,
        QDate, QDateTime, QLocale,
        QMetaObject, QObject, QPoint, QRect,
        QSize, QTime, QUrl, Qt)
    from PySide6.QtGui import (QBrush, QColor,
        QConicalGradient, QCursor,
        QFont, QFontDatabase, QGradient, QIcon,
        QImage, QKeySequence, QLinearGradient, QPainter,
        QPalette, QPixmap, QRadialGradient, QTransform)
    from PySide6.QtWidgets import (QApplication, QLabel,
        QMainWindow, QPushButton,
        QSizePolicy, QVBoxLayout, QWidget)

    class Ui_SettingsWindow(object):
        def setupUi(self, SettingsWindow):
            if not SettingsWindow.setObjectName():
                SettingsWindow.setObjectName(u"SettingsWindow")
                SettingsWindow.resize(250, 300)
                SettingsWindow.setMinimumSize(QSize(250, 300))

                self.centralwidget = QWidget(SettingsWindow)

                self.centralwidget.setObjectName(u"centralwidget")
                self.verticalLayout_2 =
                QVBoxLayout(self.centralwidget)

                self.verticalLayout_2.setObjectName(u"verticalLayout_2")
                self.labelSheetSettings =
                QLabel(self.centralwidget)

```

```

self.labelSheetSettings.setObjectName(u"labelSheetSettings")
    sizePolicy
    =
    QSizePolicy(QSizePolicy.Policy.Preferred,
    QSizePolicy.Policy.Maximum)
    sizePolicy.setHorizontalStretch(0)
    sizePolicy.setVerticalStretch(0)

sizePolicy.setHeightForWidth(self.labelSheetSettings.sizePolicy().
hasHeightForWidth())
    self.labelSheetSettings.setSizePolicy(sizePolicy)

self.labelSheetSettings.setAlignment(Qt.AlignCenter)

self.verticalLayout_2.addWidget(self.labelSheetSettings)

    self.verticalLayout = QVBoxLayout()
    self.verticalLayout.setSpacing(4)

self.verticalLayout.setObjectName(u"verticalLayout")
    self.btnSettingsShow
    =
    QPushButton(self.centralwidget)

self.btnSettingsShow.setObjectName(u"btnSettingsShow")
    sizePolicy1
    =
    QSizePolicy(QSizePolicy.Policy.Minimum,
    QSizePolicy.Policy.Expanding)
    sizePolicy1.setHorizontalStretch(0)
    sizePolicy1.setVerticalStretch(0)

sizePolicy1.setHeightForWidth(self.btnSettingsShow.sizePolicy().
hasHeightForWidth())
    self.btnSettingsShow.setSizePolicy(sizePolicy1)

self.verticalLayout.addWidget(self.btnSettingsShow)

    self.btnSettingsAdd
    =
    QPushButton(self.centralwidget)

self.btnSettingsAdd.setObjectName(u"btnSettingsAdd")

sizePolicy1.setHeightForWidth(self.btnSettingsAdd.sizePolicy().h
asHeightForWidth())
    self.btnSettingsAdd.setSizePolicy(sizePolicy1)

self.verticalLayout.addWidget(self.btnSettingsAdd)

    self.btnSettingsDelete
    =
    QPushButton(self.centralwidget)

self.btnSettingsDelete.setObjectName(u"btnSettingsDelete")

sizePolicy1.setHeightForWidth(self.btnSettingsDelete.sizePolicy().
hasHeightForWidth())
    self.btnSettingsDelete.setSizePolicy(sizePolicy1)

self.verticalLayout.addWidget(self.btnSettingsDelete)
    self.verticalLayout_2.addLayout(self.verticalLayout)
    SettingsWindow.setCentralWidget(self.centralwidget)
    self.retranslateUi(SettingsWindow)

QMetaObject.connectSlotsByName(SettingsWindow)
    # setupUi

    def retranslateUi(self, SettingsWindow):

SettingsWindow.setWindowTitle(QCoreApplication.translate("Sett
ingsWindow",
    u"\u041d\u0430\u0441\u0442\u0440\u043e\u0439\u0430\u0438",
    None))

self.labelSheetSettings.setText(QCoreApplication.translate("Settin
gsWindow",
    u"\u041d\u0430\u0440\u0430\u043c\u0435\u0442\u0440\u044b
\u0444\u043e\u0440\u043c\u0430\u0442\u0438\u0440\u043e\u04
32\u0430\u043d\u0438\u0440", None))

self.btnSettingsShow.setText(QCoreApplication.translate("Settings
Window",
    u"\u041d\u043e\u0441\u043c\u043e\u0442\u0440\u0435\u0442\u0
44c
\u043d\u0430\u0440\u0440\u0430\u043c\u0435\u0442\u0440\u044b",
    None))

self.btnSettingsAdd.setText(QCoreApplication.translate("Settings
Window",
    u"\u0414\u043e\u0431\u0430\u0432\u0438\u0442\u0440\u044c
\u043d\u0430\u0440\u0440\u0430\u043c\u0435\u0442\u0440", None))

self.btnSettingsDelete.setText(QCoreApplication.translate("Setting
sWindow",
    u"\u0423\u0434\u0430\u043b\u0438\u0438\u0442\u044c
\u043d\u0430\u0440\u0440\u0430\u043c\u0435\u0442\u0440", None))
    # retranslateUi
    Файл ui_settings_add.py

    from PySide6.QtCore import (QCoreApplication,
    QDate, QDateTime, QLocale,
    QMetaObject, QObject, QPoint, QRect,
    QSize, QTime, QUrl, Qt)

```

```

        from PySide6.QtGui import (QBrush, QColor,
        QConicalGradient, QCursor,
        QFont, QFontDatabase, QGradient, QIcon,
        QImage, QKeySequence, QLinearGradient, QPainter,
        QPalette, QPixmap, QRadialGradient, QTransform)
        from PySide6.QtWidgets import (QApplication,
        QDialog, QFormLayout, QLabel,
        QLineEdit, QPushButton, QSizePolicy, QTextEdit,
        QVBoxLayout, QWidget)

        class Ui_Dialog(object):
            def setupUi(self, Dialog):
                if not Dialog.setObjectName():
                    Dialog.setObjectName(u"Dialog")
                Dialog.resize(400, 640)
                Dialog.setMinimumSize(QSize(400, 640))
                self.verticalLayout = QVBoxLayout(Dialog)

                self.verticalLayout.setObjectName(u"verticalLayout")
                self.labelHeader = QLabel(Dialog)
                self.labelHeader.setObjectName(u"labelHeader")
                self.sizePolicy =
                QSizePolicy(QSizePolicy.Policy.Minimum,
                QSizePolicy.Policy.Expanding)
                self.sizePolicy.setHorizontalStretch(0)
                self.sizePolicy.setVerticalStretch(0)

                self.sizePolicy.setHeightForWidth(self.labelHeader.sizePolicy().hasHeightForWidth())
                self.labelHeader.setSizePolicy(self.sizePolicy)
                self.labelHeader.setAlignment(Qt.AlignCenter)

                self.verticalLayout.addWidget(self.labelHeader)

                self.formLayout = QFormLayout()
                self.formLayout.setObjectName(u"formLayout")

                self.formLayout.setFieldGrowthPolicy(QFormLayout.AllNonFixedFieldsGrow)

                self.formLayout.setRowWrapPolicy(QFormLayout.WrapAllRows)

                self.formLayout.setLabelAlignment(Qt.AlignLeading|Qt.AlignLeft|Qt.AlignVCenter)

                self.formLayout.setFormAlignment(Qt.AlignCenter)
                self.labelSheet = QLabel(Dialog)
                self.labelSheet.setObjectName(u"labelSheet")

                self.formLayout.addWidget(0,
                QFormLayout.LabelRole, self.labelSheet)

```

```

                self.labelIlocRows = QLabel(Dialog)

                self.labelIlocRows.setObjectName(u"labelIlocRows")

                self.formLayout.addWidget(1,
                QFormLayout.LabelRole, self.labelIlocRows)

                self.labelIlocColumns = QLabel(Dialog)

                self.labelIlocColumns.setObjectName(u"labelIlocColumns")

                self.formLayout.addWidget(2,
                QFormLayout.LabelRole, self.labelIlocColumns)

                self.labelDropColumn = QLabel(Dialog)

                self.labelDropColumn.setObjectName(u"labelDropColumn")

                self.formLayout.addWidget(3,
                QFormLayout.LabelRole, self.labelDropColumn)

                self.labelMidLists = QLabel(Dialog)

                self.labelMidLists.setObjectName(u"labelMidLists")

                self.formLayout.addWidget(4,
                QFormLayout.LabelRole, self.labelMidLists)

                self.labelMidNames = QLabel(Dialog)

                self.labelMidNames.setObjectName(u"labelMidNames")

                self.formLayout.addWidget(5,
                QFormLayout.LabelRole, self.labelMidNames)

                self.labelCsvPath = QLabel(Dialog)

                self.labelCsvPath.setObjectName(u"labelCsvPath")

                self.formLayout.addWidget(6,
                QFormLayout.LabelRole, self.labelCsvPath)

                self.textEditIlocRows = QTextEdit(Dialog)

                self.textEditIlocRows.setObjectName(u"textEditIlocRows")

                self.formLayout.addWidget(1,
                QFormLayout.FieldRole, self.textEditIlocRows)

                self.textEditIlocColumns = QTextEdit(Dialog)

                self.textEditIlocColumns.setObjectName(u"textEditIlocColumns")

```

```

# setupUi

self.formLayout.setWidget(2,
QFormLayout.FieldRole, self.textEditIlocColumns)

self.textEditDropColumns = QTextEdit(Dialog)

self.textEditDropColumns.setObjectName(u"textEditDropColumns")

self.formLayout.setWidget(3,
QFormLayout.FieldRole, self.textEditDropColumns)

self.textEditMidLists = QTextEdit(Dialog)

self.textEditMidLists.setObjectName(u"textEditMidLists")

self.formLayout.setWidget(4,
QFormLayout.FieldRole, self.textEditMidLists)

self.textEditMidNames = QTextEdit(Dialog)

self.textEditMidNames.setObjectName(u"textEditMidNames")

self.formLayout.setWidget(5,
QFormLayout.FieldRole, self.textEditMidNames)

self.lineEditSheet = QLineEdit(Dialog)

self.lineEditSheet.setObjectName(u"lineEditSheet")

self.formLayout.setWidget(0,
QFormLayout.FieldRole, self.lineEditSheet)

self.lineEditCSVPath = QLineEdit(Dialog)

self.lineEditCSVPath.setObjectName(u"lineEditCSVPath")

self.formLayout.setWidget(6,
QFormLayout.FieldRole, self.lineEditCSVPath)

self.verticalLayout.addLayout(self.formLayout)

self.btnSave = QPushButton(Dialog)
self.btnSave.setObjectName(u"btnSave")

self.verticalLayout.addWidget(self.btnSave)

self.retranslateUi(Dialog)

QMetaObject.connectSlotsByName(Dialog)

Dialog.setWindowTitle(QCoreApplication.translate("Dialog",
u"Dialog", None))

self.labelHeader.setText(QCoreApplication.translate("Dialog",
u"\u041d\u043e\u0432\u044b\u0439
\u043f\u0430\u0440\u0430\u043c\u0435\u0435\u0442\u0440", None))

self.labelSheet.setText(QCoreApplication.translate("Dialog",
u"\u041d\u0430\u0437\u0432\u0430\u043d\u0438\u0435
\u043b\u0438\u0441\u0442\u0430 (Sheet)", None))

self.labelIlocRows.setText(QCoreApplication.translate("Dialog",
u"\u0413\u0440\u0430\u0434\u0438\u0446\u044b
\u0441\u0442\u0440\u043e\u043a (iloc_rows)", None))

self.labelIlocColumns.setText(QCoreApplication.translate("Dialog",
u"\u0413\u0440\u0430\u0434\u0438\u0446\u044b
\u0441\u0442\u0440\u043e\u0431\u0431\u0431\u0446\u0435\u0432
(iloc_columns)", None))

self.labelDropColumn.setText(QCoreApplication.translate("Dialog",
u"\u0414\u0435\u0431\u0430\u0440\u0430\u0442\u044c
\u0442\u043e\u0431\u0430\u0446\u044b... (drop_column)",
None))

self.labelMidLists.setText(QCoreApplication.translate("Dialog",
u"\u0421\u043f\u0438\u0441\u0430\u0438
\u0430\u0430\u0442\u0435\u0433\u0433\u0435\u0440\u0438\u0439
(m_id_lists)", None))

self.labelMidNames.setText(QCoreApplication.translate("Dialog",
u"\u0414\u0430\u0437\u0432\u0430\u043d\u0438\u0435
\u0441\u0430\u0435\u0432
\u0430\u0442\u0435\u0433\u0435\u0440\u0438\u0439
(m_id_nsmes)", None))

self.labelCsvPath.setText(QCoreApplication.translate("Dialog",
u"\u0414\u0430\u0437\u0432\u0430\u043d\u0438\u0435
\u0442\u0430\u0431\u0446\u044b\u0434\u0435\u0435\u0433\u0435 csv
\u0442\u0430\u0431\u0430\u0430 (csv_path)", None))

self.textEditIlocRows.setPlaceholderText(QCoreApplication.transl
ate("Dialog",
u"\u0414\u0438\u0430\u043f\u0430\u0437\u0435\u043d\u0434
\u0441\u0442\u0440\u0440\u0435\u0430
\u0447\u0435\u0435 Enter", None))

self.textEditIlocColumns.setPlaceholderText(QCoreApplication.tra

```

```

nslate("Dialog",
u"\u0414\u0438\u0430\u043f\u0430\u0437\u043e\u043d
\u0441\u0442\u043e\u043b\u0431\u0446\u043e\u0432
\u0447\u0435\u0440 \u0435\u043d\u0442\u0440", None))

self.textEditDropColumns.setPlaceholderText(QCoreApplication.t
ranslate("Dialog", u"\u041b\u0438\u0448\u043d\u0438\u0435
\u0441\u0442\u043e\u043b\u0431\u0446\u043d\u044b
\u0447\u0435\u0440 \u0440\u0435\u0432\u0432 \u0435\u043d\u0442\u0440", None))

self.textEditMidLists.setPlaceholderText(QCoreApplication.transl
ate("Dialog", u"\u0422\u0440\u043e\u0432\u043d\u0438\u0438
\u043c\u0435\u043d\u0446\u0442\u0438\u0438\u043d\u0434\u04
35\u0430\u0441\u0430,
\u0440\u0430\u0430\u0437\u0437\u0434\u0435\u043b\u0446\u0446
\u0430\u0430\u0436\u0436\u0434\u044b\u0439
\u0443\u0440\u0440\u043e\u0432\u0432\u0435\u043d\u044c
\u0434\u0434\u0439\u043d\u044b\u043c \u0435\u043d\u0442\u0440,
\u0430\u0430\u0430\u0436\u0436\u0434\u0443\u0443\u044e
\u0430\u0430\u0430\u0442\u0435\u0433\u043e\u043e\u0440\u0438\u044e
\u043d\u0430\u0430\u0447\u0438\u043d\u0430\u0439\u0442\u0435
\u0441 \u0441\u0442\u0440\u0430\u0432\u0432", None))

self.textEditMidNames.setPlaceholderText(QCoreApplication.transl
ate("Dialog",
u"\u0414\u0430\u0437\u0437\u0432\u0430\u043d\u0438\u0438\u0446
\u0441\u0443\u0438\u0438\u0441\u0430\u043e\u043e\u0432
\u0430\u0430\u0442\u0435\u0435\u0433\u043e\u0440\u0438\u0439.
\u0441\u0441\u043e\u043b\u0438\u0438\u0447\u0435\u0441\u0441\u0442\u0432\u043e
\u0435 \u043d\u0430\u0430\u0437\u0432\u0430\u043d\u0438\u0438\u0439
\u0441\u0441\u043e\u043e\u0442\u0432\u0435\u0442\u0441\u0441\u0442\u0432\u043e
32\u0443\u0443\u0435\u0442
\u0430\u0430\u043e\u043b\u0438\u0447\u0435\u0441\u0441\u0442\u0432\u043e
43 \u0443\u0440\u0440\u043e\u0432\u0432\u043d\u0435\u0435\u0439
\u043c\u0443\u0443\u043b\u044c\u0442\u0438\u0438\u043d\u0434\u0430\u04
35\u0430\u0441\u0441\u0430 ", None))

self.lineEditSheet.setPlaceholderText(QCoreApplication.translate(
"Dialog", u"\u0422\u043e\u0447\u0447\u043d\u043e\u043e\u0435
\u043d\u0430\u0430\u0437\u0432\u0430\u043d\u0438\u0438\u0435
\u043b\u0438\u0438\u0441\u0442\u0442\u0430 \u0432 \u0441\u0441\u0441\u0441\u0441
\u0444\u0430\u0439\u043b\u0438\u0435", None))

self.lineEditCSVPath.setPlaceholderText(QCoreApplication.transl
ate("Dialog",
u"\u0414\u0430\u0430\u0437\u0432\u0430\u043d\u0438\u0438\u0435 \u0438 \u0438
\u0444\u0440\u0430\u0439\u043b\u0430 \u0441\u0441\u0441
\u0440\u0443\u0435\u0437\u0443\u0443\u044c\u0442\u0430\u0442\u0430\u0442\u043e
3\u043e\u043e", None))

self.btnSave.setText(QCoreApplication.translate("Dialog",

```

```

u"\u0421\u043e\u0445\u0440\u0440\u0430\u043d\u043d\u0438\u0442\u044c",
None))

# retranslateUi
\u0414\u0438\u0441\u0442\u0440\u043e\u0432

from PySide6.QtCore import (QCoreApplication,
QDate, QDateTime, QLocale,
QMetaObject, QObject, QPoint, QRect,
QSize, QTime, QUrl, Qt)
from PySide6.QtGui import (QBrush, QColor,
QConicalGradient, QCursor,
QFont, QFontDatabase, QGradient, QIcon,
 QImage, QKeySequence, QLinearGradient, QPainter,
QPalette, QPixmap, QRadialGradient, QTransform)
from PySide6.QtWidgets import (QApplication,
QComboBox, QDialog, QLabel,
QPushButton, QSizePolicy, QVBoxLayout,
QWidget)

class Ui_Dialog(object):
    def setupUi(self, Dialog):
        if not Dialog.setObjectName():
            Dialog.setObjectName(u"Dialog")
        Dialog.resize(250, 150)
        Dialog.setMinimumSize(QSize(250, 150))
        self.verticalLayout = QVBoxLayout(Dialog)

self.verticalLayout.setObjectName(u"verticalLayout")
        self.labelSettingsDelHeader = QLabel(Dialog)

self.labelSettingsDelHeader.setObjectName(u"labelSettingsDelHeader")

        sizePolicy =
QSizePolicy(QSizePolicy.Policy.Minimum,
QSizePolicy.Policy.Maximum)
        sizePolicy.setHorizontalStretch(0)
        sizePolicy.setVerticalStretch(0)

sizePolicy.setHeightForWidth(self.labelSettingsDelHeader.sizePol
icy().hasHeightForWidth())

self.labelSettingsDelHeader.setSizePolicy(sizePolicy)

self.labelSettingsDelHeader.setAlignment(Qt.AlignCenter)

self.verticalLayout.addWidget(self.labelSettingsDelHeader)

        self.comboBoxSelectList = QComboBox(Dialog)
        self.comboBoxSelectList.addItem("")
        self.comboBoxSelectList.addItem("")

```

```

self.comboBoxSelectList.setObjectName(u"comboBoxSelectList")

self.verticalLayout.addWidget(self.comboBoxSelectList)

        self.btnDeleteSelectedList = QPushButton(Dialog)

self.btnDeleteSelectedList.setObjectName(u"btnDeleteSelectedList
")

self.verticalLayout.addWidget(self.btnDeleteSelectedList)

        self.retranslateUi(Dialog)

        QMetaObject.connectSlotsByName(Dialog)
# setupUi

def retranslateUi(self, Dialog):

Dialog.setWindowTitle(QCoreApplication.translate("Dialog",
u"Dialog", None))

self.labelSettingsDelHeader.setText(QCoreApplication.translate("
Dialog", u"\u0423\u0434\u0430\u0430\u043b\u0438\u0442\u044c
\u043f\u0430\u0440\u0430\u043c\u0435\u0435\u0442\u0440...", None))
        self.comboBoxSelectList.setItemText(0,
QCoreApplication.translate("Dialog",
u"\u041f\u0440\u0438\u043c\u0435\u0440\u0440\u0440", None))
        self.comboBoxSelectList.setItemText(1,
QCoreApplication.translate("Dialog",
u"\u041f\u0440\u0438\u043c\u0435\u0435\u0440\u0440", None))

self.btnDeleteSelectedList.setText(QCoreApplication.translate("Di
alog", u"\u0423\u0434\u0430\u0430\u043b\u0438\u0442\u044c", None))
        # retranslateUi
        Файл ui_settings_ignore.py

        from PySide6.QtCore import (QCoreApplication,
QDate, QDateTime, QLocale,
        QMetaObject, QObject, QPoint, QRect,
        QSize, QTime, QUrl, Qt)
        from PySide6.QtGui import (QBrush, QColor,
QConicalGradient, QCursor,
        QFont, QFontDatabase, QGradient, QIcon,
        QImage, QKeySequence, QLinearGradient, QPainter,
        QPalette, QPixmap, QRadialGradient, QTransform)
        from PySide6.QtWidgets import (QAbstractItemView,
QApplication, QDialog, QLabel,

```

```

QListWidget, QListWidgetItem, QPushButton,
QSizePolicy,
QVBoxLayout, QWidget)

class Ui_Dialog(object):
    def setupUi(self, Dialog):
        if not Dialog.setObjectName():
            Dialog.setObjectName(u"Dialog")
            Dialog.resize(400, 300)
            self.verticalLayout = QVBoxLayout(Dialog)

self.verticalLayout.setObjectName(u"verticalLayout")
        self.labelSettingsIgnoreListHeader =
QLabel(Dialog)

self.labelSettingsIgnoreListHeader.setObjectName(u"labelSettings
IgnoreListHeader")

self.verticalLayout.addWidget(self.labelSettingsIgnoreListHeader)

        self.listWidgetSheetsList = QListWidget(Dialog)
        QListWidgetItem(self.listWidgetSheetsList)
        QListWidgetItem(self.listWidgetSheetsList)
        QListWidgetItem(self.listWidgetSheetsList)
        QListWidgetItem(self.listWidgetSheetsList)
        QListWidgetItem(self.listWidgetSheetsList)

self.listWidgetSheetsList.setObjectName(u"listWidgetSheetsList")

self.listWidgetSheetsList.setSelectionMode(QAbstractItemView.
MultiSelection)

self.verticalLayout.addWidget(self.listWidgetSheetsList)

        self.btnSaveSelect = QPushButton(Dialog)

self.btnSaveSelect.setObjectName(u"btnSaveSelect")

        self.verticalLayout.addWidget(self.btnSaveSelect)

        self.retranslateUi(Dialog)

        QMetaObject.connectSlotsByName(Dialog)
# setupUi

def retranslateUi(self, Dialog):

Dialog.setWindowTitle(QCoreApplication.translate("Dialog",
u"Dialog", None))

```

```

self.labelSettingsIgnoreListHeader.setText(QCoreApplication.trans
late("Dialog",
u"\u0412\u044b\u0431\u0435\u0440\u0438\u0442\u0435
\u043b\u0438\u0441\u0442\u0442\u044b
\u043a\u043e\u0442\u043e\u0440\u044b\u0435
\u043d\u0443\u0436\u043d\u043e
\u0438\u0433\u043d\u043e\u0440\u0438\u0440\u043e\u0432\u04
30\u0442\u044c...", None))

__sortingEnabled =
self.listWidgetSheetsList.isSortingEnabled()
self.listWidgetSheetsList.setSortingEnabled(False)
__qlistwidgetitem =
self.listWidgetSheetsList.item(0)

__qlistwidgetitem.setText(QCoreApplication.translate("Dialog",
u"\u041fu0440\u0438\u043c\u0435\u04401", None));
__qlistwidgetitem1 =
self.listWidgetSheetsList.item(1)

__qlistwidgetitem1.setText(QCoreApplication.translate("Dialog",
u"\u041fu0440\u0438\u043c\u0435\u04402", None));
__qlistwidgetitem2 =
self.listWidgetSheetsList.item(2)

__qlistwidgetitem2.setText(QCoreApplication.translate("Dialog",
u"\u041fu0440\u0438\u043c\u0435\u04403", None));
__qlistwidgetitem3 =
self.listWidgetSheetsList.item(3)

__qlistwidgetitem3.setText(QCoreApplication.translate("Dialog",
u"\u041fu0440\u0438\u043c\u0435\u04404", None));
__qlistwidgetitem4 =
self.listWidgetSheetsList.item(4)

__qlistwidgetitem4.setText(QCoreApplication.translate("Dialog",
u"\u041fu0440\u0438\u043c\u0435\u04405", None));

self.listWidgetSheetsList.setSortingEnabled(__sortingEnabled)

self.btnSaveSelect.setText(QCoreApplication.translate("Dialog",
u"\u0421\u043e\u0445\u0440\u0430\u043d\u0438\u0442\u044c
\u0432\u044b\u0431\u043e\u0440", None))
    # retranslateUi
    Файл JsonWindow.py
    from PySide6.QtCore import QSize
    from PySide6.QtWidgets import QMainWindow,
    QTreeView, QVBoxLayout, QWidget
    from PySide6.QtGui import QStandardItemModel,
    QStandardItem

```

```

class JsonViewer(QMainWindow):
    def __init__(self, json_data):
        super().__init__()
        self.setWindowTitle("Настройки")
        self.resize(400, 300)
        self.setMinimumSize(QSize(400, 300))

        # Основной виджет и layout
        central_widget = QWidget()
        layout = QVBoxLayout(central_widget)

        # Дерево и модель
        self.tree_view = QTreeView()
        self.model = QStandardItemModel()
        self.model.setHorizontalHeaderLabels(['Key',
'Value'])

        self.tree_view.setModel(self.model)
        layout.addWidget(self.tree_view)

        self.setCentralWidget(central_widget)

        # Заполнение модели JSON-данными
        self.load_json(json_data)

    def load_json(self, data, parent=None):
        if parent is None:
            parent = self.model.invisibleRootItem()

        if isinstance(data, dict):
            for key, value in data.items():
                key_item = QStandardItem(str(key))
                if isinstance(value, (dict, list)):
                    value_item = QStandardItem("")
                    self.load_json(value, key_item)
                else:
                    value_item = QStandardItem(str(value))
                parent.appendRow([key_item, value_item])
        elif isinstance(data, list):
            for index, item in enumerate(data):
                key_item = QStandardItem(f"[{index}]")
                if isinstance(item, (dict, list)):
                    value_item = QStandardItem("")
                    self.load_json(item, key_item)
                else:
                    value_item = QStandardItem(str(item))
                parent.appendRow([key_item, value_item])

```

Файл MainWindow.py
import json


```

import shutil
from datetime import datetime

from PySide6 import QtWidgets
from PySide6.QtWidgets import QFileDialog,
QMainWindow, QMessageBox

from config_project import Const
from sheet_format.WorkerDoFormat import
WorkerDoFormat
from sheet_format.sheet_formatting import
UlskFormatter, FullFormatter, BaseFormatter
from sheet_format.sheet_settings import get_settings
from ui.SettingsWindow import SettingsWindow
from ui.qt_designer.py_ui_files.ui_main import
Ui_MainWindow
from ui.qt_designer.py_ui_files.ui_settings_ignore
import Ui_Dialog

class MainWindow(QMainWindow):
    def __init__(self):
        super(MainWindow, self).__init__()
        self.ui = Ui_MainWindow()
        self.ui.setupUi(self)

        self.ui.radioBtnFullFile.setChecked(True)

self.ui.spinBoxFileYear.setValue(datetime.now().year)

self.ui.btnDoFormatToCSV.clicked.connect(self.start_format_async)

        self.ui.btnOpenFile.clicked.connect(self.open_file)

self.ui.btnSelectResultPath.clicked.connect(self.select_result_path)

self.ui.actionSettingsListIgnore.triggered.connect(self.open_window_settings_ignore)

self.ui.actionOpenSheetSettingsWindow.triggered.connect(self.open_window_settings)

self.ui.actionImport.triggered.connect(self.import_settings)

self.ui.actionExport.triggered.connect(self.export_settings)

self.ui.actionInsertData.toggled.connect(self.insertDataToggled)

        self.window_settings = None
        self.ignore_list = set()

```

```

def get_mode(self) -> BaseFormatter:
    if self.ui.radioBtnFullFile.isChecked():
        return FullFormatter()
    elif self.ui.radioBtnULFile.isChecked():
        return UlskFormatter()
    raise ValueError('Ни один чек бокс не выбран')

def start_format_async(self):
    self.worker = WorkerDoFormat(
        get_settings(),
        self.get_mode(),
        self.ui.textEditSelectedFilePath.toPlainText(),
        self.ui.spinBoxFileYear.value(),
        self.ui.textEditResultPath.toPlainText(),
        self.ignore_list,
        self.ui.actionInsertData.isChecked()
    )

self.worker.signal.connect(self.update_status_label)
    self.worker.start()

def update_status_label(self, text, btn_disabled):

self.ui.btnDoFormatToCSV.setDisabled(btn_disabled)
    self.ui.labelStatus.setText(text)

def open_file(self):
    file_path, _ = QFileDialog.getOpenFileName(self,
"Выберите файл", "", "Файл (*.xlsx)")
    self.ui.textEditSelectedFilePath.setText(file_path)
    if file_path else None

def select_result_path(self):
    folder_path =
QFileDialog.getExistingDirectory(self, "Выберите папку")
    self.ui.textEditResultPath.setText(folder_path)
    if folder_path else None

def open_window_settings_ignore(self):
    self.new_window = QtWidgets.QDialog()
    self.ui_settings = Ui_Dialog()
    self.ui_settings.setupUi(self.new_window)

self.new_window.setWindowTitle("Игнорируемые листы")
    self.new_window.show()

self.ui_settings.btnSaveSelect.clicked.connect(self.save_ignore_list)

        self.ui_settings.listWidgetSheetsList.clear()self.ui_settings.listWidgetSheetsList.addItem(get_settings())

def save_ignore_list(self):

```

```

        selected_items = self.ui_settings.listWidgetSheetsList.selectedItems()
        self.ignore_list = {item.text() for item in selected_items}

        QMessageBox.information(self, "Торобо!",
                                "Игнорирование задано")
        self.new_window.close()

    def open_window_settings(self):
        if self.window_settings is None or not self.window_settings.isVisible():
            self.window_settings = SettingsWindow()
            self.window_settings.show()

    def import_settings(self):
        file_path, _ = QFileDialog.getOpenFileName(self,
            "Выбрать файл", "", "JSON Files (*.json)")
        if file_path:
            try:
                with open(file_path, "r", encoding="utf-8") as f:
                    json.load(f)
                    shutil.copy(file_path,
                        Const.SETTINGS_FILE)
                    QMessageBox.information(self, "Успех",
                        "Настройки успешно импортированы!")
            except json.JSONDecodeError:
                QMessageBox.critical(self, "Ошибка",
                    "Выбранный файл не является корректным JSON!")
            except Exception as e:
                QMessageBox.critical(self, "Ошибка",
                    f"Ошибка при импорте файла: {e}")

    def export_settings(self):
        file_path, _ = QFileDialog.getSaveFileName(self,
            "Экспортировать", "", "JSON Files (*.json)")
        if file_path:
            shutil.copy(Const.SETTINGS_FILE, file_path)
            QMessageBox.information(self, "Успех",
                "Файл настроек успешно экспортирован!")

    def insertDataToggled(self, checked: bool):
        if checked:
            self.ui.btnDoFormatToCSV.setText("Обработать и загрузить в БД")
        else:
            self.ui.btnDoFormatToCSV.setText("Обработать")

```

Файл SettingsWindow.py
import json

```

from PySide6 import QtWidgets
from PySide6.QtWidgets import QMainWindow,
    QMessageBox

from sheet_format.model_settings import Setting
from sheet_format.sheet_settings import
    update_or_reset_settings, get_settings
from ui.qt_designer.py_ui_files.ui_settings import
    Ui_SettingsWindow
from ui.qt_designer.py_ui_files.ui_settings_add import
    Ui_Dialog as UIDialogAdd
from ui.qt_designer.py_ui_files.ui_settings_del import
    Ui_Dialog as UIDialogDel
from ui.JsonWindow import JsonViewer

class SettingsWindow(QMainWindow):
    def __init__(self):
        super(SettingsWindow, self).__init__()

        self.ui = Ui_SettingsWindow()
        self.ui.setupUi(self)

        self.ui.btnSettingsAdd.clicked.connect(self.btn_add)

        self.ui.btnSettingsDelete.clicked.connect(self.btn_delete)

        self.ui.btnSettingsShow.clicked.connect(self.btn_show)

        self.settings_list = get_settings()
        self.json_window = None

    def btn_add(self):
        self.add_window = QtWidgets.QDialog()
        self.add_window_ui = UIDialogAdd()
        self.add_window_ui.setupUi(self.add_window)
        self.add_window.show()
        self.add_window.setWindowTitle("Новый лист")

        self.add_window_ui.btnSave.clicked.connect(self.new_sheet_save)

    def btn_delete(self):
        self.del_window = QtWidgets.QDialog()
        self.del_window_ui = UIDialogDel()
        self.del_window_ui.setupUi(self.del_window)
        self.del_window.show()
        self.del_window.setWindowTitle("Удалить лист")

        self.del_window_ui.btnDeleteSelectedList.clicked.connect(self.delete_selected_sheet)

```

```

        self.del_window_ui.comboBoxSelectList.clear()

self.del_window_ui.comboBoxSelectList.addItem(self.settings_list)

def btn_show(self):
    try:
        with open("all_settings.json", "r",
encoding="utf-8") as f:
            data = json.load(f)
            self.json_window = JsonViewer(data)
            self.json_window.show()
    except FileNotFoundError as e:
        QMessageBox.information(self, "Настройка
нет", "Нет заданных настроек")

def new_sheet_save(self):
    new_sheet = Setting(
        sheet=self.add_window_ui.lineEditSheet.text(),
        iloc_rows=[int(item) for item in
self.add_window_ui.textEditIlocRows.toPlainText().split("\n")],
        iloc_columns=[int(item) for item in
self.add_window_ui.textEditIlocColumns.toPlainText().split("\n")],
        drop_column=[int(item) for item in
self.add_window_ui.textEditDropColumns.toPlainText().split("\n")]
    ,
        m_id_lists=[_split("\n") for _ in
self.add_window_ui.textEditMidLists.toPlainText().split("\n\n")],
        m_id_names=[_ for _ in
self.add_window_ui.textEditMidNames.toPlainText().split("\n")],

    csv_path=self.add_window_ui.lineEditCSVPath.text()
    )
    self.settings_list[new_sheet.sheet] = new_sheet
    update_or_reset_settings(self.settings_list)
    QMessageBox.information(self, "Готово!", f"Лист
\"{new_sheet.sheet}\" создан")
    self.add_window.close()

def delete_selected_sheet(self):
    selected_sheet =
self.del_window_ui.comboBoxSelectList.currentText()
    self.settings_list.pop(selected_sheet)
    update_or_reset_settings(self.settings_list)
    QMessageBox.information(self, "Готово!", f"Лист
\"{selected_sheet}\" удален")
    self.del_window.close()

```

Файл config_project.py
from pathlib import Path

```

from pydantic import ConfigDict
from pydantic_settings import BaseSettings

ROOT_DIR: Path = Path(__file__).parent
ENV_FILE_PATH: Path = ROOT_DIR / '.env'

```

```

class ConfigProject(BaseSettings):
    model_config = ConfigDict(extra='ignore')

    DB_HOST: str
    DB_PORT: int
    DB_USER: str
    DB_PASSWORD: str
    DB_NAME: str

```

```

SETTINGS_FILE: Path = ROOT_DIR /
'all_settings.json'

```

```

def DATABASE_URL_psycopg(self):
    return
f"postgresql+psycopg2://{self.DB_USER}:{self.DB_PASSWORD
}@{self.DB_HOST}:{self.DB_PORT}/{self.DB_NAME}"

```

```

Const = ConfigProject(_env_file=ENV_FILE_PATH,
_env_file_encoding='utf-8')

```

Файл main.py
import sys

from PySide6.QtWidgets import QApplication

from ui.MainWindow import MainWindow

```

if __name__ == "__main__":
    app = QApplication(sys.argv)

```

```

    window = MainWindow()
    window.show()

```

```

    sys.exit(app.exec())

```

Бакалаврская работа выполнена мной совершенно самостоятельно. Все использованные в работе материалы и концепции из опубликованной научной литературы и других источников имеют ссылки на них.

Объем работы 68 листов.

Библиография 20 наименования.

Объем приложений 15 листов.

Отпечатано в 1 экземпляре.

Один экземпляр сдан на кафедру.

« » 20 г.

_____/ _____/

(подпись)